```
package com.cleanguard.app
import android.app.Application
import com.cleanguard.app.di.AppContainer
import com.cleanguard.app.di.DefaultAppContainer
class CleanGuardApplication : Application() {
    val container: AppContainer by lazy { DefaultAppContainer(applicationContext) }
}
package com.cleanguard.app.data.apk
import android.content.ContentResolver
import android.content.Context
import android.content.pm.PackageManager
import android.os.Build
import androidx.core.net.toUri
import com.cleanguard.app.domain.apk.ApkArchiveInspector
import com.cleanguard.app.domain.apk.ApkArchiveMetadata
import java.io.File
import java.io.FileNotFoundException
import kotlinx.coroutines.CoroutineDispatcher
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.currentCoroutineContext
import kotlinx.coroutines.ensureActive
import kotlinx.coroutines.withContext
class AndroidApkArchiveInspector(
    private val context: Context,
    private val contentResolver: ContentResolver = context.contentResolver,
    private val ioDispatcher: CoroutineDispatcher = Dispatchers.IO,
) : ApkArchiveInspector {
    override suspend fun inspect(contentUri: String): ApkArchiveMetadata? = withContext(ioDispatcher) {
        val temporaryApk = File.createTempFile(TEMP_PREFIX, APK_SUFFIX, context.cacheDir)
        try {
            contentResolver.openInputStream(contentUri.toUri()).use { input ->
                if (input == null) throw FileNotFoundException("Unable to open APK URI")
                temporaryApk.outputStream().buffered().use { output ->
                    val buffer = ByteArray(STREAM_BUFFER_SIZE)
                    while (true) {
                        currentCoroutineContext().ensureActive()
                        val count = input.read(buffer)
                        if (count < 0) break
                        if (count > 0) output.write(buffer, 0, count)
                    }
                }
            }
            val packageInfo = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
                context.packageManager.getPackageArchiveInfo(
                    temporaryApk.absolutePath,
                    PackageManager.PackageInfoFlags.of(PackageManager.GET_META_DATA.toLong()),
                )
            } else {
                @Suppress("DEPRECATION")
                context.packageManager.getPackageArchiveInfo(
```

```
                    temporaryApk.absolutePath,
                    PackageManager.GET_META_DATA,
                )
            } ?: return@withContext null
            val applicationInfo = packageInfo.applicationInfo?.apply {
                sourceDir = temporaryApk.absolutePath
                publicSourceDir = temporaryApk.absolutePath
            }
            ApkArchiveMetadata(
                packageName = packageInfo.packageName,
                versionName = packageInfo.versionName,
                longVersionCode = packageInfo.longVersionCode,
                applicationLabel = applicationInfo?.let {
                    runCatching { context.packageManager.getApplicationLabel(it).toString() }.getOrNull()
                },
            )
        } finally {
            temporaryApk.delete()
        }
    }
    private companion object {
        const val TEMP_PREFIX = "cg-apk-"
        const val APK_SUFFIX = ".apk"
        const val STREAM_BUFFER_SIZE = 64 * 1024
    }
}
package com.cleanguard.app.data.apk
import android.content.pm.PackageManager
import android.os.Build
import com.cleanguard.app.domain.apk.InstalledPackageLookup
import com.cleanguard.app.domain.apk.InstalledPackageVersionProvider
class AndroidInstalledPackageVersionProvider(
    private val packageManager: PackageManager,
) : InstalledPackageVersionProvider {
    override fun lookup(packageName: String): InstalledPackageLookup = try {
        val info = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            packageManager.getPackageInfo(packageName, PackageManager.PackageInfoFlags.of(0L))
        } else {
            @Suppress("DEPRECATION")
            packageManager.getPackageInfo(packageName, 0)
        }
        InstalledPackageLookup.Installed(info.longVersionCode)
    } catch (_: PackageManager.NameNotFoundException) {
        InstalledPackageLookup.Unknown
    } catch (_: SecurityException) {
        InstalledPackageLookup.Unknown
    }
}
package com.cleanguard.app.data.apk
import com.cleanguard.app.domain.apk.ApkAnalysis
```

```
import com.cleanguard.app.domain.apk.ApkAnalyzer
import com.cleanguard.app.domain.apk.ApkArchiveInspector
import com.cleanguard.app.domain.apk.ApkCandidate
import com.cleanguard.app.domain.apk.ApkInstalledComparison
import com.cleanguard.app.domain.apk.InstalledPackageLookup
import com.cleanguard.app.domain.apk.InstalledPackageVersionProvider
import java.io.IOException
import kotlinx.coroutines.CancellationException
class ApkAnalyzerImpl(
    private val archiveInspector: ApkArchiveInspector,
    private val installedVersionProvider: InstalledPackageVersionProvider,
) : ApkAnalyzer {
    override suspend fun analyze(candidate: ApkCandidate): ApkAnalysis = try {
        val metadata = archiveInspector.inspect(candidate.contentUri)
            ?: return ApkAnalysis(
                candidate = candidate,
                metadata = null,
                installedComparison = ApkInstalledComparison.UNKNOWN,
                issueMessage = "???? APK ????",
            )
        when (val installed = installedVersionProvider.lookup(metadata.packageName)) {
            is InstalledPackageLookup.Installed -> ApkAnalysis(
                candidate = candidate,
                metadata = metadata,
                installedComparison = when {
                    metadata.longVersionCode < installed.longVersionCode ->
                        ApkInstalledComparison.OLDER_THAN_INSTALLED
                    metadata.longVersionCode == installed.longVersionCode ->
                        ApkInstalledComparison.SAME_AS_INSTALLED
                    else -> ApkInstalledComparison.NEWER_THAN_INSTALLED
                },
                installedVersionCode = installed.longVersionCode,
            )
            InstalledPackageLookup.NotInstalled -> ApkAnalysis(
                candidate = candidate,
                metadata = metadata,
                installedComparison = ApkInstalledComparison.NOT_INSTALLED,
            )
            InstalledPackageLookup.Unknown -> ApkAnalysis(
                candidate = candidate,
                metadata = metadata,
                installedComparison = ApkInstalledComparison.UNKNOWN,
                issueMessage = "? Android ??????????????????",
            )
        }
    } catch (error: Throwable) {
        if (error is CancellationException) throw error
        if (error !is IOException && error !is SecurityException && error !is IllegalArgumentException) {
            throw error
        }
```

```
        ApkAnalysis(
            candidate = candidate,
            metadata = null,
            installedComparison = ApkInstalledComparison.UNKNOWN,
            issueMessage = "APK ?????????????",
        )
    }
}
package com.cleanguard.app.data.apps
import android.app.AppOpsManager
import android.app.usage.StorageStatsManager
import android.content.Context
import android.content.Intent
import android.content.pm.ApplicationInfo
import android.content.pm.PackageManager
import android.os.Process
import android.os.storage.StorageManager
import com.cleanguard.app.domain.model.AppStorageItem
import com.cleanguard.app.domain.model.AppStorageScanResult
import com.cleanguard.app.domain.model.UsageAccessStatus
import com.cleanguard.app.domain.repository.AppStorageRepository
import java.io.IOException
import kotlinx.coroutines.CancellationException
import kotlinx.coroutines.CoroutineDispatcher
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.ensureActive
import kotlinx.coroutines.withContext
import kotlin.coroutines.coroutineContext
class AndroidAppStorageRepository(
    private val context: Context,
    private val ioDispatcher: CoroutineDispatcher = Dispatchers.IO,
) : AppStorageRepository {
    private val appOpsManager = context.getSystemService(AppOpsManager::class.java)
    private val storageStatsManager = context.getSystemService(StorageStatsManager::class.java)
    private val packageManager = context.packageManager
    override fun usageAccessStatus(): UsageAccessStatus {
        val mode = appOpsManager.checkOpNoThrow(
            AppOpsManager.OPSTR_GET_USAGE_STATS,
            Process.myUid(),
            context.packageName,
        )
        return if (mode == AppOpsManager.MODE_ALLOWED) {
            UsageAccessStatus.GRANTED
        } else {
            UsageAccessStatus.NOT_GRANTED
        }
    }
    override suspend fun loadLaunchableAppStorage(): Result<AppStorageScanResult> = withContext(ioDispatcher) {
        try {
            check(usageAccessStatus() == UsageAccessStatus.GRANTED) {
```

```
                "Usage access is required to read app storage statistics."
            }
            val launcherIntent = Intent(Intent.ACTION_MAIN).addCategory(Intent.CATEGORY_LAUNCHER)
            val launchableApps = packageManager.queryIntentActivities(
                launcherIntent,
                PackageManager.MATCH_ALL,
            ).mapNotNull { it.activityInfo?.applicationInfo }
                .distinctBy(ApplicationInfo::packageName)
            var skippedCount = 0
            val items = buildList {
                launchableApps.forEach { applicationInfo ->
                    coroutineContext.ensureActive()
                    try {
                        val stats = storageStatsManager.queryStatsForPackage(
                            StorageManager.UUID_DEFAULT,
                            applicationInfo.packageName,
                            Process.myUserHandle(),
                        )
                        add(
                            AppStorageItem(
                                packageName = applicationInfo.packageName,
                                appName = applicationInfo.loadLabel(packageManager).toString()
                                    .ifBlank { applicationInfo.packageName },
                                appBytes = stats.appBytes.coerceAtLeast(0L),
                                dataBytes = stats.dataBytes.coerceAtLeast(0L),
                                cacheBytes = stats.cacheBytes.coerceAtLeast(0L),
                            ),
                        )
                    } catch (exception: CancellationException) {
                        throw exception
                    } catch (_: PackageManager.NameNotFoundException) {
                        skippedCount += 1
                    } catch (_: SecurityException) {
                        skippedCount += 1
                    } catch (_: IOException) {
                        skippedCount += 1
                    } catch (_: IllegalArgumentException) {
                        skippedCount += 1
                    }
                }
            }
            Result.success(
                AppStorageScanResult(
                    items = items.sortedByDescending(AppStorageItem::totalBytes),
                    skippedAppCount = skippedCount,
                ),
            )
        } catch (exception: CancellationException) {
            throw exception
        } catch (exception: Exception) {
```

```
            Result.failure(exception)
        }
    }
}
package com.cleanguard.app.data.cleanup
import android.content.ContentResolver
import android.content.ContentUris
import android.content.IntentSender
import android.net.Uri
import android.os.Build
import android.provider.MediaStore
import com.cleanguard.app.domain.model.CleanupAction
import com.cleanguard.app.domain.model.CleanupOutcome
import com.cleanguard.app.domain.model.CleanupPlan
import com.cleanguard.app.domain.model.CleanupResult
import com.cleanguard.app.domain.model.ScanSource
import com.cleanguard.app.domain.model.RiskLevel
import com.cleanguard.app.domain.model.SystemConfirmationOutcome
import com.cleanguard.app.domain.repository.CleanupRepository
import com.cleanguard.app.domain.repository.CleanupHistoryRepository
import com.cleanguard.app.domain.repository.NoOpCleanupHistoryRepository
import com.cleanguard.app.domain.repository.StorageRepository
import com.cleanguard.app.domain.model.persistence.CleanupHistoryEntry
import com.cleanguard.app.domain.model.persistence.CleanupHistoryItem
import com.cleanguard.app.domain.model.persistence.CleanupHistoryType
import com.cleanguard.app.domain.model.persistence.CleanupRestoreStatus
import java.io.IOException
import kotlinx.coroutines.CancellationException
import kotlinx.coroutines.CoroutineDispatcher
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.withContext
class MediaStoreCleanupRepository(
    private val contentResolver: ContentResolver,
    private val storageRepository: StorageRepository,
    private val cleanupHistoryRepository: CleanupHistoryRepository = NoOpCleanupHistoryRepository,
    private val ioDispatcher: CoroutineDispatcher = Dispatchers.IO,
    private val currentTimeMillis: () -> Long = System::currentTimeMillis,
) : CleanupRepository {
    private val result = MutableStateFlow<CleanupResult?>(null)
    private var pendingPlan: CleanupPlan? = null
    private var undoPlan: CleanupPlan? = null
    private var beforeUsedBytes: Long = 0L
    override val latestResult: Flow<CleanupResult?> = result.asStateFlow()
    override suspend fun createTrashRequest(plan: CleanupPlan): Result<IntentSender> =
        withContext(ioDispatcher) {
            beforeUsedBytes = currentUsedBytes(fallback = 0L)
            pendingPlan = plan
```

```
            createMediaStoreTrashRequest(plan, trash = true)
        }
    override suspend fun completeTrashRequest(
        plan: CleanupPlan,
        outcome: SystemConfirmationOutcome,
    ): CleanupResult = withContext(ioDispatcher) {
        val activePlan = pendingPlan?.takeIf { it.id == plan.id } ?: plan
        val afterUsedBytes = currentUsedBytes(fallback = beforeUsedBytes)
        val completedResult = CleanupResultFactory.fromTrashOutcome(
            plan = activePlan,
            outcome = outcome,
            beforeUsedBytes = beforeUsedBytes,
            afterUsedBytes = afterUsedBytes,
        )
        undoPlan = activePlan.takeIf { outcome == SystemConfirmationOutcome.CONFIRMED }
        pendingPlan = null
        result.value = completedResult
        persistHistory(activePlan, completedResult, outcome)
        completedResult
    }
    override suspend fun createUndoRequest(): Result<IntentSender> = withContext(ioDispatcher) {
        val plan = undoPlan
            ?: return@withContext Result.failure(IllegalStateException("No trash operation is available to undo"))
        createMediaStoreTrashRequest(plan, trash = false)
    }
    override suspend fun completeUndoRequest(confirmed: Boolean): Result<CleanupResult> =
        withContext(ioDispatcher) {
            val currentResult = result.value
                ?: return@withContext Result.failure(IllegalStateException("No cleanup result is available"))
            if (!confirmed) return@withContext Result.success(currentResult)
            val updatedResult = CleanupResultFactory.afterUndo(
                result = currentResult,
                afterUsedBytes = currentUsedBytes(fallback = currentResult.beforeUsedBytes),
            )
            undoPlan = null
            result.value = updatedResult
            runCatching {
                cleanupHistoryRepository.updateRestoreStatus(
                    currentResult.cleanupId ?: return@runCatching,
                    CleanupRestoreStatus.RESTORED,
                )
            }
            Result.success(updatedResult)
        }
    private suspend fun persistHistory(
        plan: CleanupPlan,
        cleanupResult: CleanupResult,
        outcome: SystemConfirmationOutcome,
    ) {
        val entry = CleanupHistoryEntryFactory.fromTrashOutcome(
```

```
            plan = plan,
            result = cleanupResult,
            outcome = outcome,
            timestamp = currentTimeMillis(),
        )
        try {
            cleanupHistoryRepository.save(entry)
        } catch (exception: CancellationException) {
            throw exception
        } catch (_: Exception) {
        }
    }
    private fun createMediaStoreTrashRequest(
        plan: CleanupPlan,
        trash: Boolean,
    ): Result<IntentSender> {
        try {
            if (Build.VERSION.SDK_INT < Build.VERSION_CODES.R) {
                return Result.failure(
                    UnsupportedOperationException("MediaStore Trash requires Android 11 or newer"),
                )
            }
            val uris = plan.items.map { item ->
                require(item.source == ScanSource.MEDIA_STORE) {
                    "Only MediaStore items can enter a MediaStore trash request"
                }
                require(item.action == CleanupAction.TRASH) {
                    "Only TRASH actions can enter a MediaStore trash request"
                }
                require(item.riskLevel == RiskLevel.SAFE || item.riskLevel == RiskLevel.REVIEW) {
                    "Protected items cannot enter a MediaStore trash request"
                }
                mediaStoreUri(item.itemId)
                    ?: throw IllegalArgumentException("Invalid MediaStore item id: ${item.itemId}")
            }
            require(uris.isNotEmpty()) { "Trash request cannot be empty" }
            return Result.success(
                MediaStore.createTrashRequest(contentResolver, uris, trash).intentSender,
            )
        } catch (exception: CancellationException) {
            throw exception
        } catch (exception: SecurityException) {
            return Result.failure(exception)
        } catch (exception: IOException) {
            return Result.failure(exception)
        } catch (exception: IllegalArgumentException) {
            return Result.failure(exception)
        } catch (exception: IllegalStateException) {
            return Result.failure(exception)
        }
```

```
    }
    private suspend fun currentUsedBytes(fallback: Long): Long =
        storageRepository.loadStorageOverview().getOrNull()?.usedBytes ?: fallback
}
internal object CleanupResultFactory {
    fun fromTrashOutcome(
        plan: CleanupPlan,
        outcome: SystemConfirmationOutcome,
        beforeUsedBytes: Long,
        afterUsedBytes: Long,
    ): CleanupResult {
        val requestedCount = plan.items.size
        return CleanupResult(
            cleanupId = plan.id,
            requestedCount = requestedCount,
            successCount = if (outcome == SystemConfirmationOutcome.CONFIRMED) requestedCount else 0,
            failureCount = if (outcome == SystemConfirmationOutcome.FAILED) requestedCount else 0,
            cancelledCount = if (outcome == SystemConfirmationOutcome.CANCELLED) requestedCount else 0,
            reclaimedBytes = if (outcome == SystemConfirmationOutcome.CONFIRMED) {
                (beforeUsedBytes - afterUsedBytes).coerceAtLeast(0L)
            } else {
                0L
            },
            beforeUsedBytes = beforeUsedBytes.coerceAtLeast(0L),
            afterUsedBytes = afterUsedBytes.coerceAtLeast(0L),
            undoAvailable = outcome == SystemConfirmationOutcome.CONFIRMED,
            outcome = when (outcome) {
                SystemConfirmationOutcome.CONFIRMED -> CleanupOutcome.COMPLETED
                SystemConfirmationOutcome.CANCELLED -> CleanupOutcome.CANCELLED
                SystemConfirmationOutcome.FAILED -> CleanupOutcome.FAILED
            },
        )
    }
    fun afterUndo(
        result: CleanupResult,
        afterUsedBytes: Long,
    ): CleanupResult = result.copy(
        reclaimedBytes = 0L,
        afterUsedBytes = afterUsedBytes.coerceAtLeast(0L),
        undoAvailable = false,
        outcome = CleanupOutcome.UNDONE,
    )
}
internal object CleanupHistoryEntryFactory {
    fun fromTrashOutcome(
        plan: CleanupPlan,
        result: CleanupResult,
        outcome: SystemConfirmationOutcome,
        timestamp: Long,
    ): CleanupHistoryEntry = CleanupHistoryEntry(
```

```
        cleanupId = plan.id,
        timestamp = timestamp,
        requestedItemCount = result.requestedCount,
        processedItemCount = result.successCount,
        failedItemCount = result.failureCount,
        requestedBytes = plan.estimatedReclaimBytes,
        freedBytes = result.reclaimedBytes,
        cleanupType = CleanupHistoryType.MEDIA_STORE_TRASH,
        usedMediaStoreTrash = true,
        restoreStatus = if (outcome == SystemConfirmationOutcome.CONFIRMED) {
            CleanupRestoreStatus.AVAILABLE
        } else {
            CleanupRestoreStatus.NOT_AVAILABLE
        },
        items = plan.items.map { item ->
            CleanupHistoryItem(
                itemKey = item.itemId,
                contentUri = mediaStoreContentUriString(item.itemId),
            )
        },
    )
}
internal fun mediaStoreUri(itemId: String): Uri? {
    val reference = parseMediaStoreItemId(itemId) ?: return null
    val collectionUri = when (reference.collection) {
        "image" -> MediaStore.Images.Media.EXTERNAL_CONTENT_URI
        "video" -> MediaStore.Video.Media.EXTERNAL_CONTENT_URI
        "audio" -> MediaStore.Audio.Media.EXTERNAL_CONTENT_URI
        else -> return null
    }
    return ContentUris.withAppendedId(collectionUri, reference.mediaId)
}
internal fun mediaStoreContentUriString(itemId: String): String? {
    val reference = parseMediaStoreItemId(itemId) ?: return null
    val path = when (reference.collection) {
        "image" -> "images"
        "video" -> "video"
        "audio" -> "audio"
        else -> return null
    }
    return "content://media/external/$path/media/${reference.mediaId}"
}
internal data class MediaStoreItemReference(
    val collection: String,
    val mediaId: Long,
)
internal fun parseMediaStoreItemId(itemId: String): MediaStoreItemReference? {
    val match = MEDIA_ITEM_ID.matchEntire(itemId) ?: return null
    val mediaId = match.groupValues[2].toLongOrNull() ?: return null
    return MediaStoreItemReference(match.groupValues[1], mediaId)
```

```
}
private val MEDIA_ITEM_ID = Regex("^media:(image|video|audio):([0-9]+)$")
package com.cleanguard.app.data.cleanup
import android.content.ContentResolver
import android.content.IntentSender
import android.net.Uri
import android.os.Build
import android.provider.MediaStore
import androidx.annotation.RequiresApi
import com.cleanguard.app.domain.model.SystemConfirmationOutcome
import com.cleanguard.app.domain.model.persistence.CleanupHistoryEntry
import com.cleanguard.app.domain.model.persistence.CleanupHistoryType
import com.cleanguard.app.domain.model.persistence.CleanupRestoreStatus
import com.cleanguard.app.domain.repository.CleanupHistoryRepository
import com.cleanguard.app.domain.repository.MediaRecoveryRepository
import java.io.IOException
import kotlinx.coroutines.CancellationException
import kotlinx.coroutines.CoroutineDispatcher
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
class MediaStoreRecoveryRepository internal constructor(
    private val historyRepository: CleanupHistoryRepository,
    private val ioDispatcher: CoroutineDispatcher = Dispatchers.IO,
    private val sdkInt: Int = Build.VERSION.SDK_INT,
    private val requestFactory: MediaStoreTrashRequestFactory,
) : MediaRecoveryRepository {
    constructor(
        contentResolver: ContentResolver,
        historyRepository: CleanupHistoryRepository,
        ioDispatcher: CoroutineDispatcher = Dispatchers.IO,
    ) : this(
        historyRepository = historyRepository,
        ioDispatcher = ioDispatcher,
        sdkInt = Build.VERSION.SDK_INT,
        requestFactory = AndroidMediaStoreTrashRequestFactory(contentResolver),
    )
    override suspend fun createRestoreRequest(
        entry: CleanupHistoryEntry,
    ): Result<IntentSender> = withContext(ioDispatcher) {
        if (entry.restoreStatus != CleanupRestoreStatus.AVAILABLE) {
            return@withContext Result.failure(
                IllegalStateException("This cleanup history entry is not available for restore"),
            )
        }
        try {
            if (sdkInt < Build.VERSION_CODES.R) {
                throw UnsupportedOperationException("MediaStore restore requires Android 11 or newer")
            }
            Result.success(requestFactory.create(requireRestoreUris(entry), trash = false))
        } catch (exception: CancellationException) {
```

```
            throw exception
        } catch (exception: Exception) {
            if (exception is SecurityException ||
                exception is IOException ||
                exception is IllegalArgumentException ||
                exception is IllegalStateException ||
                exception is UnsupportedOperationException
            ) {
                markUnavailable(entry.cleanupId)
                Result.failure(exception)
            } else {
                throw exception
            }
        }
    }
    override suspend fun completeRestore(
        entry: CleanupHistoryEntry,
        outcome: SystemConfirmationOutcome,
    ): Result<Unit> = withContext(ioDispatcher) {
        try {
            when (outcome) {
                SystemConfirmationOutcome.CONFIRMED -> updateStatus(
                    entry.cleanupId,
                    CleanupRestoreStatus.RESTORED,
                )
                SystemConfirmationOutcome.CANCELLED -> Unit
                SystemConfirmationOutcome.FAILED -> markUnavailable(entry.cleanupId)
            }
            Result.success(Unit)
        } catch (exception: CancellationException) {
            throw exception
        } catch (exception: Exception) {
            Result.failure(exception)
        }
    }
    private suspend fun markUnavailable(cleanupId: String) {
        try {
            updateStatus(cleanupId, CleanupRestoreStatus.FAILED)
        } catch (exception: CancellationException) {
            throw exception
        } catch (_: Exception) {
        }
    }
    private suspend fun updateStatus(cleanupId: String, status: CleanupRestoreStatus) {
        check(historyRepository.updateRestoreStatus(cleanupId, status) == 1) {
            "Cleanup history entry no longer exists"
        }
    }
}
internal fun requireRestoreUris(entry: CleanupHistoryEntry): List<Uri> {
```

```
    return restoreUriStrings(entry).map(Uri::parse)
}
internal fun restoreUriStrings(entry: CleanupHistoryEntry): List<String> {
    require(entry.cleanupType == CleanupHistoryType.MEDIA_STORE_TRASH)
    require(entry.usedMediaStoreTrash)
    require(entry.processedItemCount > 0)
    val uris = entry.items.map { item ->
        val uri = item.contentUri
            ?: throw IllegalArgumentException("Missing MediaStore URI")
        require(uri.startsWith("${ContentResolver.SCHEME_CONTENT}://media/")) {
            "Invalid MediaStore URI"
        }
        uri
    }.distinct()
    require(uris.isNotEmpty()) { "Restore request cannot be empty" }
    return uris
}
internal fun interface MediaStoreTrashRequestFactory {
    fun create(uris: List<Uri>, trash: Boolean): IntentSender
}
private class AndroidMediaStoreTrashRequestFactory(
    private val contentResolver: ContentResolver,
) : MediaStoreTrashRequestFactory {
    @RequiresApi(Build.VERSION_CODES.R)
    override fun create(uris: List<Uri>, trash: Boolean): IntentSender =
        MediaStore.createTrashRequest(contentResolver, uris, trash).intentSender
}
package com.cleanguard.app.data.download
import com.cleanguard.app.domain.download.DownloadAgeBucket
import com.cleanguard.app.domain.download.DownloadAgeInsight
import com.cleanguard.app.domain.download.DownloadInsightsEngine
import com.cleanguard.app.domain.model.ScannedFile
import com.cleanguard.app.domain.model.ScanSource
class DownloadInsightsEngineImpl : DownloadInsightsEngine {
    override fun analyze(files: List<ScannedFile>, nowEpochMillis: Long): List<DownloadAgeInsight> {
        val grouped = files.asSequence()
            .filter { it.source == ScanSource.SAF }
            .groupBy { it.ageBucket(nowEpochMillis) }
        return DownloadAgeBucket.entries.map { bucket ->
            val bucketFiles = grouped[bucket].orEmpty().sortedWith(
                compareByDescending<ScannedFile> { it.sizeBytes }.thenBy { it.id },
            )
            DownloadAgeInsight(
                bucket = bucket,
                files = bucketFiles,
                fileCount = bucketFiles.size,
                totalBytes = bucketFiles.fold(0L) { total, file -> saturatingAdd(total, file.sizeBytes) },
                largestFile = bucketFiles.maxByOrNull { it.sizeBytes },
                fileTypes = bucketFiles.mapTo(sortedSetOf()) { file -> file.typeLabel() },
            )
```

```
        }
    }
}
private fun ScannedFile.ageBucket(now: Long): DownloadAgeBucket {
    if (modifiedAtEpochMillis <= 0L) return DownloadAgeBucket.UNKNOWN
    val age = (now - modifiedAtEpochMillis).coerceAtLeast(0L)
    return when {
        age < DAYS_30 -> DownloadAgeBucket.RECENT_30_DAYS
        age < DAYS_90 -> DownloadAgeBucket.DAYS_30_TO_90
        age < DAYS_180 -> DownloadAgeBucket.DAYS_90_TO_180
        else -> DownloadAgeBucket.OVER_180_DAYS
    }
}
private fun ScannedFile.typeLabel(): String = mimeType
    ?.substringBefore('/')
    ?.takeIf { it.isNotBlank() }
    ?: category.name.lowercase()
private fun saturatingAdd(total: Long, value: Long): Long =
    if (value > 0L && total > Long.MAX_VALUE - value) Long.MAX_VALUE else total + value
private const val DAY_MILLIS = 24L * 60L * 60L * 1_000L
private const val DAYS_30 = 30L * DAY_MILLIS
private const val DAYS_90 = 90L * DAY_MILLIS
private const val DAYS_180 = 180L * DAY_MILLIS
package com.cleanguard.app.data.duplicate
import com.cleanguard.app.domain.duplicate.DuplicateFile
import com.cleanguard.app.domain.duplicate.DuplicateFileHasher
import com.cleanguard.app.domain.model.persistence.HashCacheEntry
import com.cleanguard.app.domain.repository.HashCacheRepository
class CachingDuplicateFileHasher(
    private val cacheRepository: HashCacheRepository,
    private val calculator: StreamHashCalculator,
    private val currentTimeMillis: () -> Long = System::currentTimeMillis,
) : DuplicateFileHasher {
    override suspend fun quickHash(file: DuplicateFile): String {
        val cached = currentEntry(file)
        cached?.quickHash?.let { return it }
        val hash = calculator.quickHash(file)
        cacheRepository.upsert(
            cached.orNew(file).copy(
                quickHash = hash,
                updatedAt = currentTimeMillis(),
            ),
        )
        return hash
    }
    override suspend fun fullHash(file: DuplicateFile): String {
        val cached = currentEntry(file)
        cached?.fullHash?.let { return it }
        val hash = calculator.fullHash(file)
        cacheRepository.upsert(
```

```
            cached.orNew(file).copy(
                fullHash = hash,
                updatedAt = currentTimeMillis(),
            ),
        )
        return hash
    }
    private suspend fun currentEntry(file: DuplicateFile): HashCacheEntry? {
        cacheRepository.invalidateStale(
            contentUri = file.contentUri,
            sizeBytes = file.sizeBytes,
            modifiedAt = file.modifiedAtEpochMillis,
        )
        return cacheRepository.findCurrent(
            contentUri = file.contentUri,
            sizeBytes = file.sizeBytes,
            modifiedAt = file.modifiedAtEpochMillis,
        )
    }
    private fun HashCacheEntry?.orNew(file: DuplicateFile): HashCacheEntry = this ?: HashCacheEntry(
        id = "${file.contentUri}#${file.sizeBytes}#${file.modifiedAtEpochMillis}",
        contentUri = file.contentUri,
        sizeBytes = file.sizeBytes,
        modifiedAt = file.modifiedAtEpochMillis,
        quickHash = null,
        fullHash = null,
        updatedAt = currentTimeMillis(),
    )
}
package com.cleanguard.app.data.duplicate
import com.cleanguard.app.domain.duplicate.DuplicateGroup
import com.cleanguard.app.domain.duplicate.DuplicateSessionRepository
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
class CurrentDuplicateSessionRepository : DuplicateSessionRepository {
    private val mutableGroups = MutableStateFlow<List<DuplicateGroup>>(emptyList())
    private var cleanupSelection: Set<String> = emptySet()
    override val groups: StateFlow<List<DuplicateGroup>> = mutableGroups.asStateFlow()
    override suspend fun replaceGroups(groups: List<DuplicateGroup>) {
        mutableGroups.value = groups.toList()
    }
    override fun findGroup(groupId: String): DuplicateGroup? =
        mutableGroups.value.firstOrNull { it.id == groupId }
    override fun stageCleanupSelection(itemIds: Set<String>) {
        cleanupSelection = itemIds.toSet()
    }
    override fun consumeCleanupSelection(): Set<String> = cleanupSelection.also {
        cleanupSelection = emptySet()
    }
```

```
}
package com.cleanguard.app.data.duplicate
import com.cleanguard.app.domain.duplicate.DuplicateFile
import com.cleanguard.app.domain.duplicate.DuplicateFileHasher
import com.cleanguard.app.domain.duplicate.DuplicateGroup
import com.cleanguard.app.domain.duplicate.DuplicateVerificationEngine
import com.cleanguard.app.domain.duplicate.DuplicateVerificationIssue
import com.cleanguard.app.domain.duplicate.DuplicateVerificationResult
import com.cleanguard.app.domain.duplicate.DuplicateVerificationStatus
import com.cleanguard.app.domain.model.RiskLevel
import java.io.IOException
import java.util.Collections
import java.util.concurrent.atomic.AtomicInteger
import kotlinx.coroutines.CancellationException
import kotlinx.coroutines.async
import kotlinx.coroutines.awaitAll
import kotlinx.coroutines.coroutineScope
import kotlinx.coroutines.currentCoroutineContext
import kotlinx.coroutines.ensureActive
import kotlinx.coroutines.yield
class DuplicateVerificationEngineImpl(
    private val fileHasher: DuplicateFileHasher,
    fullHashParallelism: Int = MAX_FULL_HASH_PARALLELISM,
) : DuplicateVerificationEngine {
    private val fullHashParallelism = fullHashParallelism.coerceIn(1, MAX_FULL_HASH_PARALLELISM)
    override suspend fun verify(files: List<DuplicateFile>): DuplicateVerificationResult {
        val issues = mutableListOf<DuplicateVerificationIssue>()
        val sizeGroups = files.asSequence()
            .filter { it.sizeBytes > 0L && it.riskLevel == RiskLevel.REVIEW }
            .groupBy(DuplicateFile::sizeBytes)
            .values
            .filter { it.size >= 2 }
        val quickGroups = buildList {
            sizeGroups.forEach { sameSizeFiles ->
                val quickHashes = ArrayList<Pair<String, DuplicateFile>>(sameSizeFiles.size)
                sameSizeFiles.forEachIndexed { index, file ->
                    if (index % CANCELLATION_CHECK_INTERVAL == 0) yield()
                    runHash(file, issues) { fileHasher.quickHash(file) }
                        ?.let { quickHashes += it to file }
                }
                quickHashes.groupBy(keySelector = { it.first }, valueTransform = { it.second })
                    .filterValues { it.size >= 2 }
                    .forEach { (quickHash, groupedFiles) -> add(quickHash to groupedFiles) }
            }
        }
        val confirmedGroups = quickGroups.flatMap { (quickHash, groupedFiles) ->
            val fullHashes = hashFullWithBoundedWorkers(groupedFiles, issues)
            fullHashes.groupBy(keySelector = { it.first }, valueTransform = { it.second })
                .filterValues { it.size >= 2 }
                .map { (fullHash, filesWithSameContent) ->
```

```
                    val sortedFiles = filesWithSameContent.sortedWith(
                        compareByDescending<DuplicateFile> { it.modifiedAtEpochMillis }
                            .thenBy(DuplicateFile::itemId),
                    )
                    val totalBytes = saturatingMultiply(sortedFiles.first().sizeBytes, sortedFiles.size)
                    DuplicateGroup(
                        id = "duplicate:$fullHash",
                        files = sortedFiles,
                        totalBytes = totalBytes,
                        reclaimableBytes = saturatingMultiply(
                            sortedFiles.first().sizeBytes,
                            sortedFiles.size - 1,
                        ),
                        verificationStatus = DuplicateVerificationStatus.CONFIRMED_DUPLICATE,
                        quickHash = quickHash,
                        fullHash = fullHash,
                    )
                }
        }.sortedWith(compareByDescending<DuplicateGroup>(DuplicateGroup::reclaimableBytes).thenBy { it.id })
        return DuplicateVerificationResult(
            groups = confirmedGroups,
            issues = issues.distinctBy { it.itemId },
        )
    }
    private suspend fun hashFullWithBoundedWorkers(
        files: List<DuplicateFile>,
        issues: MutableList<DuplicateVerificationIssue>,
    ): List<Pair<String, DuplicateFile>> = coroutineScope {
        val nextIndex = AtomicInteger(0)
        val results = Collections.synchronizedList(
            ArrayList<Pair<String, DuplicateFile>>(files.size),
        )
        List(minOf(fullHashParallelism, files.size)) {
            async {
                while (true) {
                    currentCoroutineContext().ensureActive()
                    val index = nextIndex.getAndIncrement()
                    if (index >= files.size) break
                    if (index % CANCELLATION_CHECK_INTERVAL == 0) yield()
                    val file = files[index]
                    runHash(file, issues) { fileHasher.fullHash(file) }
                        ?.let { hash -> results += hash to file }
                }
            }
        }.awaitAll()
        results.toList()
    }
    private suspend fun runHash(
        file: DuplicateFile,
        issues: MutableList<DuplicateVerificationIssue>,
```

```
        block: suspend () -> String,
    ): String? = try {
        block()
    } catch (error: Throwable) {
        if (error is CancellationException) throw error
        if (error !is IOException && error !is SecurityException && error !is IllegalArgumentException) {
            throw error
        }
        synchronized(issues) {
            issues += DuplicateVerificationIssue(
                itemId = file.itemId,
                message = "??????????????????????",
            )
        }
        null
    }
    companion object {
        const val MAX_FULL_HASH_PARALLELISM = 2
    }
}
private const val CANCELLATION_CHECK_INTERVAL = 64
private fun saturatingMultiply(value: Long, multiplier: Int): Long {
    if (value <= 0L || multiplier <= 0) return 0L
    return if (value > Long.MAX_VALUE / multiplier) Long.MAX_VALUE else value * multiplier
}
package com.cleanguard.app.data.duplicate
import android.content.ContentResolver
import androidx.core.net.toUri
import com.cleanguard.app.domain.duplicate.DuplicateFile
import java.io.FileNotFoundException
import java.io.InputStream
import java.nio.ByteBuffer
import java.security.MessageDigest
import kotlinx.coroutines.CoroutineDispatcher
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.currentCoroutineContext
import kotlinx.coroutines.ensureActive
import kotlinx.coroutines.withContext
fun interface HashInputSource {
    fun open(contentUri: String): InputStream
}
class ContentResolverHashInputSource(
    private val contentResolver: ContentResolver,
) : HashInputSource {
    override fun open(contentUri: String): InputStream =
        contentResolver.openInputStream(contentUri.toUri())
            ?: throw FileNotFoundException("Unable to open content URI")
}
class StreamHashCalculator(
    private val inputSource: HashInputSource,
```

```
    private val ioDispatcher: CoroutineDispatcher = Dispatchers.IO,
) {
    suspend fun quickHash(file: DuplicateFile): String = withContext(ioDispatcher) {
        val digest = MessageDigest.getInstance(HASH_ALGORITHM)
        digest.update(ByteBuffer.allocate(Long.SIZE_BYTES).putLong(file.sizeBytes).array())
        inputSource.open(file.contentUri).use { stream ->
            digest.update(stream.readAtMost(QUICK_HASH_CHUNK_SIZE))
        }
        if (file.sizeBytes > QUICK_HASH_CHUNK_SIZE) {
            inputSource.open(file.contentUri).use { stream ->
                stream.skipFully((file.sizeBytes - QUICK_HASH_CHUNK_SIZE).coerceAtLeast(0L))
                digest.update(stream.readAtMost(QUICK_HASH_CHUNK_SIZE))
            }
        }
        digest.digest().toHex()
    }
    suspend fun fullHash(file: DuplicateFile): String = withContext(ioDispatcher) {
        val digest = MessageDigest.getInstance(HASH_ALGORITHM)
        val buffer = ByteArray(STREAM_BUFFER_SIZE)
        inputSource.open(file.contentUri).use { stream ->
            while (true) {
                currentCoroutineContext().ensureActive()
                val count = stream.read(buffer)
                if (count < 0) break
                if (count > 0) digest.update(buffer, 0, count)
            }
        }
        digest.digest().toHex()
    }
    private suspend fun InputStream.skipFully(byteCount: Long) {
        var remaining = byteCount
        val fallback = ByteArray(STREAM_BUFFER_SIZE)
        while (remaining > 0L) {
            currentCoroutineContext().ensureActive()
            val skipped = skip(remaining)
            if (skipped > 0L) {
                remaining -= skipped
            } else {
                val read = read(fallback, 0, minOf(fallback.size.toLong(), remaining).toInt())
                if (read < 0) break
                remaining -= read
            }
        }
    }
    private suspend fun InputStream.readAtMost(maxBytes: Int): ByteArray {
        val output = ByteArray(maxBytes)
        var offset = 0
        while (offset < maxBytes) {
            currentCoroutineContext().ensureActive()
            val count = read(output, offset, maxBytes - offset)
```

```
            if (count < 0) break
            offset += count
        }
        return if (offset == output.size) output else output.copyOf(offset)
    }
    private fun ByteArray.toHex(): String = joinToString(separator = "") { byte -> "%02x".format(byte) }
    companion object {
        const val QUICK_HASH_CHUNK_SIZE = 64 * 1024
        private const val STREAM_BUFFER_SIZE = 64 * 1024
        private const val HASH_ALGORITHM = "SHA-256"
    }
}
package com.cleanguard.app.data.local
import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase
import com.cleanguard.app.data.local.dao.AppStorageHistoryDao
import com.cleanguard.app.data.local.dao.CleanupHistoryDao
import com.cleanguard.app.data.local.dao.HashCacheDao
import com.cleanguard.app.data.local.dao.IgnoreRuleDao
import com.cleanguard.app.data.local.dao.IndexedItemDao
import com.cleanguard.app.data.local.dao.MediaSyncStateDao
import com.cleanguard.app.data.local.dao.ScanHistoryDao
import com.cleanguard.app.data.local.entity.AppStorageSnapshotEntity
import com.cleanguard.app.data.local.entity.CleanupHistoryEntity
import com.cleanguard.app.data.local.entity.CleanupHistoryItemEntity
import com.cleanguard.app.data.local.entity.HashCacheEntity
import com.cleanguard.app.data.local.entity.IgnoredItemEntity
import com.cleanguard.app.data.local.entity.IndexedItemEntity
import com.cleanguard.app.data.local.entity.MediaSyncStateEntity
import com.cleanguard.app.data.local.entity.ScanSnapshotEntity
@Database(
    entities = [
        IndexedItemEntity::class,
        MediaSyncStateEntity::class,
        ScanSnapshotEntity::class,
        AppStorageSnapshotEntity::class,
        HashCacheEntity::class,
        IgnoredItemEntity::class,
        CleanupHistoryEntity::class,
        CleanupHistoryItemEntity::class,
    ],
    version = CleanGuardDatabase.VERSION,
    exportSchema = true,
)
abstract class CleanGuardDatabase : RoomDatabase() {
    abstract fun indexedItemDao(): IndexedItemDao
    abstract fun mediaSyncStateDao(): MediaSyncStateDao
    abstract fun scanHistoryDao(): ScanHistoryDao
```

```
    abstract fun appStorageHistoryDao(): AppStorageHistoryDao
    abstract fun hashCacheDao(): HashCacheDao
    abstract fun ignoreRuleDao(): IgnoreRuleDao
    abstract fun cleanupHistoryDao(): CleanupHistoryDao
    companion object {
        const val VERSION = 1
        const val DATABASE_NAME = "cleanguard.db"
        @Volatile
        private var instance: CleanGuardDatabase? = null
        fun getInstance(context: Context): CleanGuardDatabase =
            instance ?: synchronized(this) {
                instance ?: Room.databaseBuilder(
                    context.applicationContext,
                    CleanGuardDatabase::class.java,
                    DATABASE_NAME,
                ).build().also { instance = it }
            }
    }
}
package com.cleanguard.app.data.local.dao
import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import com.cleanguard.app.data.local.entity.AppStorageSnapshotEntity
import kotlinx.coroutines.flow.Flow
data class AppStorageTotalRow(
    val timestamp: Long,
    val totalBytes: Long,
)
@Dao
interface AppStorageHistoryDao {
    @Insert(onConflict = OnConflictStrategy.ABORT)
    suspend fun insert(snapshot: AppStorageSnapshotEntity): Long
    @Insert(onConflict = OnConflictStrategy.ABORT)
    suspend fun insertAll(snapshots: List<AppStorageSnapshotEntity>): List<Long>
    @Query("SELECT MAX(timestamp) FROM app_storage_snapshots")
    suspend fun findLatestTimestamp(): Long?
    @Query(
        "SELECT * FROM app_storage_snapshots " +
            "WHERE timestamp = :timestamp ORDER BY totalBytes DESC, packageName ASC",
    )
    suspend fun findAtTimestamp(timestamp: Long): List<AppStorageSnapshotEntity>
    @Query(
        "SELECT * FROM app_storage_snapshots " +
            "WHERE packageName = :packageName ORDER BY timestamp DESC",
    )
    fun observeForPackage(packageName: String): Flow<List<AppStorageSnapshotEntity>>
    @Query(
        "SELECT timestamp, SUM(totalBytes) AS totalBytes FROM app_storage_snapshots " +
```

```
            "GROUP BY timestamp ORDER BY timestamp DESC LIMIT :limit",
    )
    fun observeRecentTotals(limit: Int): Flow<List<AppStorageTotalRow>>
    @Query("DELETE FROM app_storage_snapshots WHERE timestamp < :cutoffEpochMillis")
    suspend fun deleteBefore(cutoffEpochMillis: Long): Int
}
package com.cleanguard.app.data.local.dao
import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import androidx.room.Transaction
import com.cleanguard.app.data.local.entity.CleanupHistoryEntity
import com.cleanguard.app.data.local.entity.CleanupHistoryItemEntity
import com.cleanguard.app.data.local.relation.CleanupHistoryWithItems
import kotlinx.coroutines.flow.Flow
@Dao
interface CleanupHistoryDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun upsertHistory(history: CleanupHistoryEntity)
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun upsertItems(items: List<CleanupHistoryItemEntity>)
    @Query("DELETE FROM cleanup_history_items WHERE cleanupId = :cleanupId")
    suspend fun deleteItems(cleanupId: String)
    @Transaction
    suspend fun save(
        history: CleanupHistoryEntity,
        items: List<CleanupHistoryItemEntity>,
    ) {
        upsertHistory(history)
        deleteItems(history.cleanupId)
        if (items.isNotEmpty()) upsertItems(items)
    }
    @Transaction
    @Query("SELECT * FROM cleanup_history WHERE cleanupId = :cleanupId LIMIT 1")
    suspend fun findById(cleanupId: String): CleanupHistoryWithItems?
    @Transaction
    @Query("SELECT * FROM cleanup_history ORDER BY timestamp DESC LIMIT :limit")
    fun observeRecent(limit: Int): Flow<List<CleanupHistoryWithItems>>
    @Query("UPDATE cleanup_history SET restoreStatus = :restoreStatus WHERE cleanupId = :cleanupId")
    suspend fun updateRestoreStatus(cleanupId: String, restoreStatus: String): Int
    @Query("SELECT COUNT(*) FROM cleanup_history")
    suspend fun count(): Int
}
package com.cleanguard.app.data.local.dao
import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import com.cleanguard.app.data.local.entity.HashCacheEntity
```

```
@Dao
interface HashCacheDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun upsert(entry: HashCacheEntity)
    @Query(
        "SELECT * FROM hash_cache WHERE contentUri = :contentUri " +
            "AND sizeBytes = :sizeBytes AND modifiedAt = :modifiedAt LIMIT 1",
    )
    suspend fun findCurrent(
        contentUri: String,
        sizeBytes: Long,
        modifiedAt: Long,
    ): HashCacheEntity?
    @Query(
        "DELETE FROM hash_cache WHERE contentUri = :contentUri " +
            "AND (sizeBytes != :sizeBytes OR modifiedAt != :modifiedAt)",
    )
    suspend fun invalidateStale(
        contentUri: String,
        sizeBytes: Long,
        modifiedAt: Long,
    ): Int
    @Query("SELECT COUNT(*) FROM hash_cache WHERE contentUri = :contentUri")
    suspend fun countForUri(contentUri: String): Int
    @Query(
        "DELETE FROM hash_cache WHERE updatedAt < :cutoffEpochMillis AND NOT EXISTS (" +
            "SELECT 1 FROM indexed_items WHERE indexed_items.contentUri = hash_cache.contentUri " +
            "AND indexed_items.sizeBytes = hash_cache.sizeBytes " +
            "AND indexed_items.modifiedAt = hash_cache.modifiedAt)",
    )
    suspend fun pruneOrphansUpdatedBefore(cutoffEpochMillis: Long): Int
}
package com.cleanguard.app.data.local.dao
import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import com.cleanguard.app.data.local.entity.IgnoredItemEntity
import kotlinx.coroutines.flow.Flow
@Dao
interface IgnoreRuleDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun upsert(item: IgnoredItemEntity)
    @Query(
        "SELECT * FROM ignored_items WHERE expiresAt IS NULL OR expiresAt > :nowEpochMillis " +
            "ORDER BY ignoredAt DESC",
    )
    fun observeActive(nowEpochMillis: Long): Flow<List<IgnoredItemEntity>>
    @Query("DELETE FROM ignored_items WHERE itemKey = :itemKey")
    suspend fun deleteByItemKey(itemKey: String): Int
```

```
    @Query("SELECT COUNT(*) FROM ignored_items")
    suspend fun count(): Int
    @Query("DELETE FROM ignored_items WHERE expiresAt IS NOT NULL AND expiresAt <= :nowEpochMillis")
    suspend fun deleteExpired(nowEpochMillis: Long): Int
    @Query("DELETE FROM ignored_items")
    suspend fun deleteAll(): Int
}
package com.cleanguard.app.data.local.dao
import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import androidx.room.Transaction
import com.cleanguard.app.data.local.entity.IndexedItemEntity
import kotlinx.coroutines.flow.Flow
@Dao
interface IndexedItemDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun upsert(item: IndexedItemEntity)
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun upsertAll(items: List<IndexedItemEntity>)
    @Transaction
    suspend fun upsertScanBatch(
        scanId: String,
        items: List<IndexedItemEntity>,
    ) {
        upsertAll(items.map { it.copy(lastSeenScanId = scanId) })
    }
    @Query("SELECT * FROM indexed_items WHERE contentUri = :contentUri LIMIT 1")
    suspend fun findByContentUri(contentUri: String): IndexedItemEntity?
    @Query("SELECT * FROM indexed_items WHERE volumeName = :volumeName ORDER BY displayName")
    fun observeByVolume(volumeName: String): Flow<List<IndexedItemEntity>>
    @Query("SELECT * FROM indexed_items WHERE volumeName = :volumeName ORDER BY id")
    suspend fun findByVolume(volumeName: String): List<IndexedItemEntity>
    @Query("SELECT * FROM indexed_items ORDER BY volumeName, id")
    suspend fun findAll(): List<IndexedItemEntity>
    @Query("UPDATE indexed_items SET lastSeenScanId = :scanId WHERE volumeName = :volumeName")
    suspend fun markVolumeSeen(volumeName: String, scanId: String): Int
    @Query("DELETE FROM indexed_items WHERE id IN (:itemIds)")
    suspend fun deleteByIds(itemIds: List<String>): Int
    @Query("DELETE FROM indexed_items WHERE volumeName = :volumeName")
    suspend fun deleteByVolume(volumeName: String): Int
    @Query("DELETE FROM indexed_items WHERE contentUri = :contentUri")
    suspend fun deleteByContentUri(contentUri: String): Int
    @Query("SELECT COUNT(*) FROM indexed_items")
    suspend fun count(): Int
}
package com.cleanguard.app.data.local.dao
import androidx.room.Dao
import androidx.room.Insert
```

```
import androidx.room.OnConflictStrategy
import androidx.room.Query
import com.cleanguard.app.data.local.entity.MediaSyncStateEntity
import kotlinx.coroutines.flow.Flow
@Dao
interface MediaSyncStateDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun upsert(state: MediaSyncStateEntity)
    @Query("SELECT * FROM media_sync_states WHERE volumeName = :volumeName LIMIT 1")
    suspend fun findByVolume(volumeName: String): MediaSyncStateEntity?
    @Query("SELECT * FROM media_sync_states ORDER BY volumeName")
    fun observeAll(): Flow<List<MediaSyncStateEntity>>
    @Query("SELECT * FROM media_sync_states ORDER BY volumeName")
    suspend fun findAll(): List<MediaSyncStateEntity>
    @Query("DELETE FROM media_sync_states WHERE volumeName = :volumeName")
    suspend fun deleteByVolume(volumeName: String): Int
    @Query("SELECT COUNT(*) FROM media_sync_states")
    suspend fun count(): Int
}
package com.cleanguard.app.data.local.dao
import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import com.cleanguard.app.data.local.entity.ScanSnapshotEntity
import kotlinx.coroutines.flow.Flow
@Dao
interface ScanHistoryDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun upsert(snapshot: ScanSnapshotEntity)
    @Query("SELECT * FROM scan_snapshots WHERE scanId = :scanId LIMIT 1")
    suspend fun findById(scanId: String): ScanSnapshotEntity?
    @Query(
        "SELECT * FROM scan_snapshots " +
            "WHERE scanStatus = 'COMPLETED' " +
            "ORDER BY completedAt DESC, startedAt DESC LIMIT :limit",
    )
    fun observeRecent(limit: Int): Flow<List<ScanSnapshotEntity>>
    @Query(
        "DELETE FROM scan_snapshots WHERE scanStatus = 'COMPLETED' AND scanId NOT IN (" +
            "SELECT scanId FROM scan_snapshots WHERE scanStatus = 'COMPLETED' " +
            "ORDER BY completedAt DESC, startedAt DESC LIMIT :retainCount" +
            ")",
    )
    suspend fun pruneCompleted(retainCount: Int): Int
    @Query("SELECT COUNT(*) FROM scan_snapshots")
    suspend fun count(): Int
}
package com.cleanguard.app.data.local.entity
import androidx.room.Entity
```

```
import androidx.room.Index
import androidx.room.PrimaryKey
@Entity(
    tableName = "app_storage_snapshots",
    indices = [
        Index(value = ["packageName", "timestamp"]),
        Index(value = ["timestamp"]),
    ],
)
data class AppStorageSnapshotEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val packageName: String,
    val appName: String,
    val timestamp: Long,
    val appBytes: Long,
    val dataBytes: Long,
    val cacheBytes: Long,
    val totalBytes: Long,
)
package com.cleanguard.app.data.local.entity
import androidx.room.Entity
import androidx.room.Index
import androidx.room.PrimaryKey
@Entity(
    tableName = "cleanup_history",
    indices = [Index(value = ["timestamp"])],
)
data class CleanupHistoryEntity(
    @PrimaryKey val cleanupId: String,
    val timestamp: Long,
    val requestedItemCount: Int,
    val processedItemCount: Int,
    val failedItemCount: Int,
    val requestedBytes: Long,
    val freedBytes: Long,
    val cleanupType: String,
    val usedMediaStoreTrash: Boolean,
    val restoreStatus: String,
)
package com.cleanguard.app.data.local.entity
import androidx.room.Entity
import androidx.room.ForeignKey
import androidx.room.Index
@Entity(
    tableName = "cleanup_history_items",
    primaryKeys = ["cleanupId", "itemKey"],
    foreignKeys = [
        ForeignKey(
            entity = CleanupHistoryEntity::class,
            parentColumns = ["cleanupId"],
```

```
            childColumns = ["cleanupId"],
            onDelete = ForeignKey.CASCADE,
        ),
    ],
    indices = [Index(value = ["cleanupId"])],
)
data class CleanupHistoryItemEntity(
    val cleanupId: String,
    val itemKey: String,
    val contentUri: String?,
)
package com.cleanguard.app.data.local.entity
import androidx.room.Entity
import androidx.room.Index
import androidx.room.PrimaryKey
@Entity(
    tableName = "hash_cache",
    indices = [
        Index(value = ["contentUri", "sizeBytes", "modifiedAt"], unique = true),
        Index(value = ["quickHash"]),
        Index(value = ["fullHash"]),
    ],
)
data class HashCacheEntity(
    @PrimaryKey val id: String,
    val contentUri: String,
    val sizeBytes: Long,
    val modifiedAt: Long,
    val quickHash: String?,
    val fullHash: String?,
    val updatedAt: Long,
)
package com.cleanguard.app.data.local.entity
import androidx.room.Entity
import androidx.room.Index
import androidx.room.PrimaryKey
@Entity(
    tableName = "ignored_items",
    indices = [
        Index(value = ["itemKey"], unique = true),
        Index(value = ["expiresAt"]),
    ],
)
data class IgnoredItemEntity(
    @PrimaryKey val id: String,
    val itemKey: String,
    val ignoredAt: Long,
    val expiresAt: Long?,
    val ignoreType: String,
)
```

```
package com.cleanguard.app.data.local.entity
import androidx.room.Entity
import androidx.room.Index
import androidx.room.PrimaryKey
@Entity(
    tableName = "indexed_items",
    indices = [
        Index(value = ["contentUri"], unique = true),
        Index(value = ["volumeName"]),
        Index(value = ["lastSeenScanId"]),
        Index(value = ["sizeBytes", "modifiedAt"]),
        Index(value = ["quickHash"]),
        Index(value = ["fullHash"]),
    ],
)
data class IndexedItemEntity(
    @PrimaryKey val id: String,
    val contentUri: String,
    val source: String,
    val displayName: String,
    val mimeType: String?,
    val sizeBytes: Long,
    val modifiedAt: Long,
    val relativePath: String?,
    val category: String,
    val riskLevel: String,
    val quickHash: String?,
    val fullHash: String?,
    val mediaStoreGeneration: Long?,
    val lastSeenScanId: String?,
    val lastIndexedAt: Long,
    val volumeName: String?,
)
package com.cleanguard.app.data.local.entity
import androidx.room.Entity
import androidx.room.PrimaryKey
@Entity(tableName = "media_sync_states")
data class MediaSyncStateEntity(
    @PrimaryKey val volumeName: String,
    val mediaStoreVersion: String?,
    val lastGeneration: Long?,
    val lastSuccessfulScanAt: Long?,
)
package com.cleanguard.app.data.local.entity
import androidx.room.Entity
import androidx.room.Index
import androidx.room.PrimaryKey
@Entity(
    tableName = "scan_snapshots",
    indices = [Index(value = ["startedAt"])],
```

```
)
data class ScanSnapshotEntity(
    @PrimaryKey val scanId: String,
    val startedAt: Long,
    val completedAt: Long?,
    val durationMs: Long?,
    val scanStatus: String,
    val usedBytes: Long,
    val freeBytes: Long,
    val totalBytes: Long,
    val imageBytes: Long,
    val videoBytes: Long,
    val audioBytes: Long,
    val downloadBytes: Long,
    val apkBytes: Long,
    val largeFileBytes: Long,
    val duplicateCandidateBytes: Long,
    val safeBytes: Long,
    val reviewBytes: Long,
    val dangerBytes: Long,
    val unknownBytes: Long,
    val scannedItemCount: Int,
    val isIncremental: Boolean,
    val mediaAccessState: String,
)
package com.cleanguard.app.data.local.mapper
import com.cleanguard.app.data.local.entity.AppStorageSnapshotEntity
import com.cleanguard.app.data.local.entity.CleanupHistoryEntity
import com.cleanguard.app.data.local.entity.CleanupHistoryItemEntity
import com.cleanguard.app.data.local.entity.HashCacheEntity
import com.cleanguard.app.data.local.entity.IgnoredItemEntity
import com.cleanguard.app.data.local.entity.IndexedItemEntity
import com.cleanguard.app.data.local.entity.MediaSyncStateEntity
import com.cleanguard.app.data.local.entity.ScanSnapshotEntity
import com.cleanguard.app.data.local.relation.CleanupHistoryWithItems
import com.cleanguard.app.domain.model.FileCategory
import com.cleanguard.app.domain.model.RiskLevel
import com.cleanguard.app.domain.model.ScanSource
import com.cleanguard.app.domain.model.persistence.AppStorageSnapshot
import com.cleanguard.app.domain.model.persistence.CleanupHistoryEntry
import com.cleanguard.app.domain.model.persistence.CleanupHistoryItem
import com.cleanguard.app.domain.model.persistence.CleanupHistoryType
import com.cleanguard.app.domain.model.persistence.CleanupRestoreStatus
import com.cleanguard.app.domain.model.persistence.HashCacheEntry
import com.cleanguard.app.domain.model.persistence.IgnoreType
import com.cleanguard.app.domain.model.persistence.IgnoredItem
import com.cleanguard.app.domain.model.persistence.IndexedItemRecord
import com.cleanguard.app.domain.model.persistence.MediaSyncState
import com.cleanguard.app.domain.model.persistence.PersistedMediaAccessState
import com.cleanguard.app.domain.model.persistence.PersistedScanStatus
```

```
import com.cleanguard.app.domain.model.persistence.ScanSnapshot
fun IndexedItemRecord.toEntity(): IndexedItemEntity = IndexedItemEntity(
    id = id,
    contentUri = contentUri,
    source = source.name,
    displayName = displayName,
    mimeType = mimeType,
    sizeBytes = sizeBytes,
    modifiedAt = modifiedAt,
    relativePath = relativePath,
    category = category.name,
    riskLevel = riskLevel.name,
    quickHash = quickHash,
    fullHash = fullHash,
    mediaStoreGeneration = mediaStoreGeneration,
    lastSeenScanId = lastSeenScanId,
    lastIndexedAt = lastIndexedAt,
    volumeName = volumeName,
)
fun IndexedItemEntity.toDomain(): IndexedItemRecord = IndexedItemRecord(
    id = id,
    contentUri = contentUri,
    source = ScanSource.valueOf(source),
    displayName = displayName,
    mimeType = mimeType,
    sizeBytes = sizeBytes,
    modifiedAt = modifiedAt,
    relativePath = relativePath,
    category = FileCategory.valueOf(category),
    riskLevel = RiskLevel.valueOf(riskLevel),
    quickHash = quickHash,
    fullHash = fullHash,
    mediaStoreGeneration = mediaStoreGeneration,
    lastSeenScanId = lastSeenScanId,
    lastIndexedAt = lastIndexedAt,
    volumeName = volumeName,
)
fun MediaSyncState.toEntity() = MediaSyncStateEntity(
    volumeName = volumeName,
    mediaStoreVersion = mediaStoreVersion,
    lastGeneration = lastGeneration,
    lastSuccessfulScanAt = lastSuccessfulScanAt,
)
fun MediaSyncStateEntity.toDomain() = MediaSyncState(
    volumeName = volumeName,
    mediaStoreVersion = mediaStoreVersion,
    lastGeneration = lastGeneration,
    lastSuccessfulScanAt = lastSuccessfulScanAt,
)
fun ScanSnapshot.toEntity() = ScanSnapshotEntity(
```

```
                            appStorageTotals = timeline.appStorageTotals.associate {
                                it.timestamp to it.totalBytes
                            },
                            hasError = false,
                        )
                    }
                }
            }
        }
        fun onEvent(event: ScanHistoryUiEvent) {
            when (event) {
                is ScanHistoryUiEvent.SelectMetric ->
                    _uiState.update { it.copy(selectedMetric = event.metric) }
            }
        }
}
package com.cleanguard.app.feature.scanning
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.Button
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.lifecycle.compose.collectAsStateWithLifecycle
import androidx.compose.ui.res.stringResource
import com.cleanguard.app.R
import com.cleanguard.app.ui.components.CleanGuardTopBar
import com.cleanguard.app.ui.components.InfoCard
import com.cleanguard.app.ui.components.MetricTile
import com.cleanguard.app.ui.util.formatBytes
import com.cleanguard.app.ui.util.labelResource
import kotlin.math.roundToInt
```

```
@Composable
fun ScanningRoute(
    viewModel: ScanningViewModel,
    onBack: () -> Unit,
    onViewResults: () -> Unit,
) {
    val state by viewModel.uiState.collectAsStateWithLifecycle()
    ScanningScreen(
        uiState = state,
        onBack = onBack,
        onEvent = { event ->
            if (event == ScanningUiEvent.ViewResults) onViewResults() else viewModel.onEvent(event)
        },
    )
}
@Composable
fun ScanningScreen(
    uiState: ScanningUiState,
    onBack: () -> Unit,
    onEvent: (ScanningUiEvent) -> Unit,
) {
    Scaffold(topBar = { CleanGuardTopBar("????", onBack) }) { innerPadding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(innerPadding)
                .verticalScroll(rememberScrollState())
                .padding(20.dp),
            verticalArrangement = Arrangement.spacedBy(20.dp),
        ) {
            Box(modifier = Modifier.fillMaxWidth(), contentAlignment = Alignment.Center) {
                CircularProgressIndicator(
                    progress = { uiState.progress },
                    modifier = Modifier.size(164.dp),
                    strokeWidth = 12.dp,
                    trackColor = MaterialTheme.colorScheme.surfaceVariant,
                )
                Column(horizontalAlignment = Alignment.CenterHorizontally) {
                    Text(
                        "${(uiState.progress * 100).roundToInt()}%",
                        style = MaterialTheme.typography.displaySmall,
                    )
                    Text(uiState.statusLabel())
                }
            }
            Column(verticalArrangement = Arrangement.spacedBy(5.dp)) {
                Text(uiState.stage, style = MaterialTheme.typography.titleLarge)
                Text(
                    uiState.statusMessage,
                    color = MaterialTheme.colorScheme.onSurfaceVariant,
```

```
            )
        }
        InfoCard("????") {
            Row(horizontalArrangement = Arrangement.spacedBy(10.dp)) {
                MetricTile(
                    stringResource(R.string.scan_type_label),
                    stringResource(uiState.scanExecutionType.labelResource()),
                    Modifier.weight(1f),
                )
                MetricTile(
                    stringResource(R.string.scan_duration_label),
                    stringResource(R.string.scan_duration_value, uiState.scanDurationMs / 1_000.0),
                    Modifier.weight(1f),
                )
            }
            Row(horizontalArrangement = Arrangement.spacedBy(10.dp)) {
                MetricTile("?????", uiState.scannedFileCount.toString(), Modifier.weight(1f))
                MetricTile("?????", uiState.scannedBytes.formatBytes(), Modifier.weight(1f))
            }
            Row(horizontalArrangement = Arrangement.spacedBy(10.dp)) {
                MetricTile("???????", uiState.classifiedFileCount.toString(), Modifier.weight(1f))
                MetricTile("????", uiState.errorCount.toString(), Modifier.weight(1f))
            }
            MetricTile("????????", uiState.largeFileBytes.formatBytes(), Modifier.fillMaxWidth())
            if (uiState.isComplete) {
                Row(horizontalArrangement = Arrangement.spacedBy(10.dp)) {
                    MetricTile(
                        stringResource(R.string.scan_changed_label),
                        uiState.changedItemCount.toString(),
                        Modifier.weight(1f),
                    )
                    MetricTile(
                        stringResource(R.string.scan_removed_label),
                        uiState.removedItemCount.toString(),
                        Modifier.weight(1f),
                    )
                }
            }
        }
        if (uiState.isScanning) {
            OutlinedButton(
                onClick = { onEvent(ScanningUiEvent.StopScan) },
                modifier = Modifier.fillMaxWidth(),
            ) {
                Text("????", fontWeight = FontWeight.SemiBold)
            }
        } else if (uiState.isStopped || uiState.failureMessage != null || uiState.requiresPermission) {
            OutlinedButton(
                onClick = { onEvent(ScanningUiEvent.RetryScan) },
                modifier = Modifier.fillMaxWidth(),
```

```
                ) {
                    Text(if (uiState.requiresPermission) "??????" else "????")
                }
            } else if (uiState.isComplete) {
                OutlinedButton(
                    onClick = { onEvent(ScanningUiEvent.RetryScan) },
                    modifier = Modifier.fillMaxWidth(),
                ) {
                    Text("????")
                }
                Button(
                    onClick = { onEvent(ScanningUiEvent.ViewResults) },
                    modifier = Modifier.fillMaxWidth(),
                ) {
                    Text("??????")
                }
                Text(
                    "??????????????????????????????????",
                    style = MaterialTheme.typography.bodyMedium,
                    color = MaterialTheme.colorScheme.onSurfaceVariant,
                )
            }
            Spacer(Modifier.height(8.dp))
        }
    }
}
private fun ScanningUiState.statusLabel(): String = when {
    requiresPermission -> "???"
    isStopped -> "???"
    isComplete -> "???"
    isScanning -> "?????"
    failureMessage != null -> "????"
    else -> "???"
}
package com.cleanguard.app.feature.scanning
sealed interface ScanningUiEvent {
    data object StopScan : ScanningUiEvent
    data object RetryScan : ScanningUiEvent
    data object ViewResults : ScanningUiEvent
}
package com.cleanguard.app.feature.scanning
import com.cleanguard.app.domain.model.MediaScanExecutionType
data class ScanningUiState(
    val stage: String = "????????",
    val statusMessage: String = "??????????????????????????????",
    val progress: Float = 0f,
    val scannedFileCount: Long = 0L,
    val scannedBytes: Long = 0L,
    val classifiedFileCount: Long = 0L,
    val largeFileBytes: Long = 0L,
```

```
    val errorCount: Int = 0,
    val isScanning: Boolean = false,
    val isComplete: Boolean = false,
    val isStopped: Boolean = false,
    val requiresPermission: Boolean = false,
    val failureMessage: String? = null,
    val scanExecutionType: MediaScanExecutionType = MediaScanExecutionType.FULL,
    val changedItemCount: Int = 0,
    val removedItemCount: Int = 0,
    val scanDurationMs: Long = 0L,
)
package com.cleanguard.app.feature.scanning
import android.os.Trace
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.cleanguard.app.domain.model.MediaPermissionStatus
import com.cleanguard.app.domain.model.FileCategory
import com.cleanguard.app.domain.model.LargeFileThreshold
import com.cleanguard.app.domain.model.MediaScanEvent
import com.cleanguard.app.domain.model.MediaScanMetadata
import com.cleanguard.app.domain.model.MediaScanStage
import com.cleanguard.app.domain.model.SafScanEvent
import com.cleanguard.app.domain.model.ScannedFile
import com.cleanguard.app.domain.model.ScanRunInfo
import com.cleanguard.app.domain.model.toScannedFile
import com.cleanguard.app.domain.repository.DownloadAccessRepository
import com.cleanguard.app.domain.repository.PermissionRepository
import com.cleanguard.app.domain.repository.SafScanner
import com.cleanguard.app.domain.repository.ScanSessionRepository
import com.cleanguard.app.domain.repository.SettingsRepository
import com.cleanguard.app.domain.usecase.ScanMediaUseCase
import com.cleanguard.app.domain.usecase.ApplySafetyRulesUseCase
import kotlinx.coroutines.CancellationException
import kotlinx.coroutines.Job
import kotlinx.coroutines.currentCoroutineContext
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.collect
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.job
import kotlinx.coroutines.launch
import java.util.concurrent.atomic.AtomicInteger
class ScanningViewModel(
    private val scanMedia: ScanMediaUseCase,
    private val applySafetyRules: ApplySafetyRulesUseCase,
    private val permissionRepository: PermissionRepository,
    private val scanSessionRepository: ScanSessionRepository,
    private val downloadAccessRepository: DownloadAccessRepository,
```

```
    private val safScanner: SafScanner,
    private val settingsRepository: SettingsRepository? = null,
) : ViewModel() {
    private val _uiState = MutableStateFlow(ScanningUiState())
    val uiState: StateFlow<ScanningUiState> = _uiState.asStateFlow()
    private var scanJob: Job? = null
    private val scanTraceCookie = AtomicInteger()
    init {
        startScan()
    }
    fun onEvent(event: ScanningUiEvent) {
        when (event) {
            ScanningUiEvent.StopScan -> stopScan()
            ScanningUiEvent.RetryScan -> startScan()
            ScanningUiEvent.ViewResults -> Unit
        }
    }
    private fun startScan() {
        scanJob?.cancel()
        _uiState.value = ScanningUiState(isScanning = true)
        val traceCookie = scanTraceCookie.incrementAndGet()
        val isTraceStarted = beginScanTrace(traceCookie)
        scanJob = viewModelScope.launch {
            try {
                val hasMediaAccess = permissionRepository.getMediaPermissionStatus() != MediaPermissionStatus.NONE
                val treeUri = downloadAccessRepository.persistedTreeUri()
                if (!hasMediaAccess && treeUri == null) {
                    _uiState.value = ScanningUiState(
                        stage = "????????",
                        statusMessage = "???????????????? Download ?????????",
                        requiresPermission = true,
                    )
                    scanJob = null
                    return@launch
                }
                val scannedFiles = mutableListOf<ScannedFile>()
                var scannedBytes = 0L
                var errorCount = 0
                var mediaFileCount = 0L
                var mediaBytes = 0L
                var mediaScanMetadata = MediaScanMetadata()
                if (hasMediaAccess) {
                    scanMedia().collect { event ->
                        when (event) {
                            is MediaScanEvent.Progress -> {
                                val progress = event.value
                                _uiState.update { state ->
                                    state.copy(
                                        stage = progress.stage.displayName(),
                                        statusMessage = "??????????????????",
```

```
                                progress = progress.fraction * if (treeUri == null) 0.85f else 0.65f,
                                scannedFileCount = progress.scannedFileCount,
                                scannedBytes = progress.scannedBytes,
                                errorCount = progress.errorCount,
                                isScanning = true,
                                isComplete = false,
                                failureMessage = null,
                                scanExecutionType = progress.executionType,
                            )
                        }
                    }
                    is MediaScanEvent.Completed -> {
                        val mediaFiles = event.result.items.map { it.toScannedFile() }
                        scannedFiles += mediaFiles
                        mediaFileCount = mediaFiles.size.toLong()
                        mediaBytes = event.result.totalBytes
                        scannedBytes = mediaBytes
                        errorCount = event.result.errors.size
                        mediaScanMetadata = event.result.metadata
                    }
                }
            }
        }
        if (treeUri != null) {
            safScanner.scan(treeUri).collect { event ->
                when (event) {
                    is SafScanEvent.Progress -> {
                        val progress = event.value
                        _uiState.update {
                            it.copy(
                                stage = "???????????",
                                statusMessage = "?????????????????????",
                                progress = if (hasMediaAccess) 0.78f else 0.72f,
                                scannedFileCount = mediaFileCount + progress.scannedFileCount,
                                scannedBytes = saturatingAdd(mediaBytes, progress.scannedBytes),
                                errorCount = errorCount + progress.errorCount,
                                isScanning = true,
                                isComplete = false,
                            )
                        }
                    }
                    is SafScanEvent.Completed -> {
                        scannedFiles += event.result.items
                        scannedBytes = saturatingAdd(mediaBytes, event.result.totalBytes)
                        errorCount += event.result.errors.size
                    }
                }
            }
        }
        _uiState.update {
```

```
                    it.copy(
                        stage = "????????",
                        statusMessage = "??????????????????????????",
                        progress = 0.92f,
                        scannedFileCount = scannedFiles.size.toLong(),
                        scannedBytes = scannedBytes,
                        errorCount = errorCount,
                    )
                }
                val threshold = settingsRepository?.settings?.first()?.largeFileThreshold
                    ?: LargeFileThreshold.DEFAULT
                val classifiedResult = applySafetyRules(scannedFiles, threshold)
                if (classifiedResult.isFailure) {
                    _uiState.update {
                        it.copy(
                            stage = "??????????",
                            statusMessage = "??????????????????????????",
                            isScanning = false,
                            isComplete = false,
                            failureMessage = "????????",
                        )
                    }
                    return@launch
                }
                val classifiedItems = classifiedResult.getOrThrow()
                scanSessionRepository.replaceScanSession(
                    scannedFiles = scannedFiles,
                    analyzedItems = classifiedItems,
                    threshold = threshold,
                    scanRunInfo = ScanRunInfo(
                        executionType = mediaScanMetadata.executionType,
                        durationMs = mediaScanMetadata.durationMs,
                        scannedItemCount = scannedFiles.size,
                        changedItemCount = mediaScanMetadata.changedItemCount,
                        removedItemCount = mediaScanMetadata.removedItemCount,
                        scannedVolumeCount = mediaScanMetadata.scannedVolumeCount,
                    ),
                )
                val largeFileBytes = classifiedItems
                    .filter { it.category == FileCategory.LARGE_FILE }
                    .sumOf { it.sizeBytes }
                _uiState.update {
                    it.copy(
                        stage = "????????",
                        statusMessage = if (errorCount == 0) {
                            "??????????????????"
                        } else {
                            "????????$errorCount ?????????"
                        },
                        progress = 1f,
```

```
                        scannedFileCount = scannedFiles.size.toLong(),
                        scannedBytes = scannedBytes,
                        classifiedFileCount = classifiedItems.size.toLong(),
                        largeFileBytes = largeFileBytes,
                        errorCount = errorCount,
                        isScanning = false,
                        isComplete = true,
                        scanExecutionType = mediaScanMetadata.executionType,
                        changedItemCount = mediaScanMetadata.changedItemCount,
                        removedItemCount = mediaScanMetadata.removedItemCount,
                        scanDurationMs = mediaScanMetadata.durationMs,
                    )
                }
                if (currentCoroutineContext().job === scanJob) {
                    scanJob = null
                }
            } catch (cancelled: CancellationException) {
                if (currentCoroutineContext().job === scanJob) {
                    scanJob = null
                    _uiState.update {
                        it.copy(
                            stage = "?????",
                            statusMessage = "??????????????????????",
                            isScanning = false,
                            isComplete = false,
                            isStopped = true,
                        )
                    }
                }
                throw cancelled
            } catch (_: Exception) {
                if (currentCoroutineContext().job === scanJob) {
                    scanJob = null
                    _uiState.update {
                        it.copy(
                            stage = "????????",
                            statusMessage = "??????????????????????",
                            isScanning = false,
                            isComplete = false,
                            failureMessage = "????",
                        )
                    }
                }
            } finally {
                if (isTraceStarted) endScanTrace(traceCookie)
            }
        }
    }
    private fun stopScan() {
        val activeJob = scanJob ?: return
```

```
        scanJob = null
        activeJob.cancel(CancellationException("????????"))
        _uiState.update {
            it.copy(
                stage = "?????",
                statusMessage = "??????????????????????",
                isScanning = false,
                isComplete = false,
                isStopped = true,
            )
        }
    }
}
private const val SCAN_TRACE_SECTION = "CleanGuardScan"
private fun beginScanTrace(cookie: Int): Boolean = try {
    Trace.beginAsyncSection(SCAN_TRACE_SECTION, cookie)
    true
} catch (_: RuntimeException) {
    false
}
private fun endScanTrace(cookie: Int) {
    try {
        Trace.endAsyncSection(SCAN_TRACE_SECTION, cookie)
    } catch (_: RuntimeException) {
    }
}
private fun saturatingAdd(left: Long, right: Long): Long =
    if (right > 0L && left > Long.MAX_VALUE - right) Long.MAX_VALUE else left + right
private fun MediaScanStage.displayName(): String = when (this) {
    MediaScanStage.PREPARING -> "????????"
    MediaScanStage.IMAGES -> "??????"
    MediaScanStage.VIDEOS -> "??????"
    MediaScanStage.AUDIO -> "??????"
    MediaScanStage.COMPLETE -> "????????"
}
package com.cleanguard.app.feature.settings
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.horizontalScroll
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.FilterChip
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
```

```
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.unit.dp
import androidx.lifecycle.compose.collectAsStateWithLifecycle
import com.cleanguard.app.R
import com.cleanguard.app.domain.model.LargeFileThreshold
import com.cleanguard.app.domain.model.persistence.ScanHistoryRetentionPolicy
import com.cleanguard.app.ui.components.CleanGuardTopBar
import com.cleanguard.app.ui.components.InfoCard
@Composable
fun SettingsRoute(
    viewModel: SettingsViewModel,
    onBack: () -> Unit,
    onOpenPermissionCenter: () -> Unit,
) {
    val state by viewModel.uiState.collectAsStateWithLifecycle()
    SettingsScreen(state, onBack, onOpenPermissionCenter, viewModel::onEvent)
}
@Composable
fun SettingsScreen(
    uiState: SettingsUiState,
    onBack: () -> Unit,
    onOpenPermissionCenter: () -> Unit,
    onEvent: (SettingsUiEvent) -> Unit,
) {
    Scaffold(topBar = { CleanGuardTopBar(stringResource(R.string.settings_title), onBack) }) { padding ->
        Column(
            Modifier.fillMaxSize().padding(padding).verticalScroll(rememberScrollState()).padding(20.dp),
            verticalArrangement = Arrangement.spacedBy(16.dp),
        ) {
            InfoCard(stringResource(R.string.settings_scan_title)) {
                Text(stringResource(R.string.settings_large_file_threshold_help), color = MaterialTheme.colorScheme.onSurfaceVa
                ChoiceRow(LargeFileThreshold.entries, uiState.largeFileThreshold, { it.displayName }) {
                    onEvent(SettingsUiEvent.SelectLargeFileThreshold(it))
                }
            }
            InfoCard(stringResource(R.string.settings_duplicate_title)) {
                Text(stringResource(R.string.settings_duplicate_quick))
                Text(stringResource(R.string.settings_duplicate_full), color = MaterialTheme.colorScheme.onSurfaceVariant)
                Text(stringResource(R.string.settings_duplicate_no_auto), color = MaterialTheme.colorScheme.onSurfaceVariant)
            }
            InfoCard(stringResource(R.string.settings_history_title)) {
                Text(stringResource(R.string.settings_history_help), color = MaterialTheme.colorScheme.onSurfaceVariant)
                ChoiceRow(
                    ScanHistoryRetentionPolicy.entries,
                    uiState.historyRetentionPolicy,
                    { policy ->
                        when (policy) {
```

```
                            ScanHistoryRetentionPolicy.TEN_SCANS -> stringResource(R.string.settings_history_10)
                            ScanHistoryRetentionPolicy.THIRTY_SCANS -> stringResource(R.string.settings_history_30)
                            ScanHistoryRetentionPolicy.NINETY_SCANS -> stringResource(R.string.settings_history_90)
                            ScanHistoryRetentionPolicy.UNLIMITED -> stringResource(R.string.settings_history_all)
                        }
                    },
                ) { onEvent(SettingsUiEvent.SelectHistoryRetention(it)) }
            }
            InfoCard(stringResource(R.string.settings_permissions_title)) {
                Text(stringResource(R.string.settings_permissions_help), color = MaterialTheme.colorScheme.onSurfaceVariant)
                OutlinedButton(onClick = onOpenPermissionCenter, modifier = Modifier.fillMaxWidth()) {
                    Text(stringResource(R.string.settings_open_permission_center))
                }
            }
            InfoCard(stringResource(R.string.settings_ignore_title)) {
                Text(stringResource(R.string.settings_ignore_count, uiState.ignoredItemCount))
                Text(stringResource(R.string.settings_ignore_help), color = MaterialTheme.colorScheme.onSurfaceVariant)
                OutlinedButton(
                    onClick = { onEvent(SettingsUiEvent.ClearIgnoredItems) },
                    enabled = uiState.ignoredItemCount > 0,
                    modifier = Modifier.fillMaxWidth(),
                ) { Text(stringResource(R.string.settings_ignore_clear)) }
            }
            InfoCard(stringResource(R.string.settings_privacy_title)) {
                Text(stringResource(R.string.privacy_local_only))
                Text(stringResource(R.string.privacy_no_upload))
                Text(stringResource(R.string.privacy_read_scope))
                Text(stringResource(R.string.privacy_hash_disclosure))
                Text(stringResource(R.string.privacy_no_chat_content))
                Text(stringResource(R.string.privacy_no_private_dirs))
            }
        }
    }
}
@Composable
private fun <T> ChoiceRow(
    choices: List<T>,
    selected: T,
    label: @Composable (T) -> String,
    onSelect: (T) -> Unit,
) {
    Row(
        Modifier.fillMaxWidth().horizontalScroll(rememberScrollState()),
        horizontalArrangement = Arrangement.spacedBy(8.dp),
    ) {
        choices.forEach { choice ->
            FilterChip(
                selected = choice == selected,
                onClick = { onSelect(choice) },
                label = { Text(label(choice)) },
```

```
            )
        }
    }
}
package com.cleanguard.app.feature.settings
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.cleanguard.app.domain.model.LargeFileThreshold
import com.cleanguard.app.domain.model.persistence.ScanHistoryRetentionPolicy
import com.cleanguard.app.domain.repository.IgnoreRuleRepository
import com.cleanguard.app.domain.repository.ScanHistoryRepository
import com.cleanguard.app.domain.repository.SettingsRepository
import kotlinx.coroutines.flow.SharingStarted
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.combine
import kotlinx.coroutines.flow.stateIn
import kotlinx.coroutines.launch
data class SettingsUiState(
    val largeFileThreshold: LargeFileThreshold = LargeFileThreshold.DEFAULT,
    val historyRetentionPolicy: ScanHistoryRetentionPolicy = ScanHistoryRetentionPolicy.DEFAULT,
    val ignoredItemCount: Int = 0,
)
sealed interface SettingsUiEvent {
    data class SelectLargeFileThreshold(val value: LargeFileThreshold) : SettingsUiEvent
    data class SelectHistoryRetention(val value: ScanHistoryRetentionPolicy) : SettingsUiEvent
    data object ClearIgnoredItems : SettingsUiEvent
}
class SettingsViewModel(
    private val settingsRepository: SettingsRepository,
    private val scanHistoryRepository: ScanHistoryRepository,
    private val ignoreRuleRepository: IgnoreRuleRepository,
    private val currentTimeMillis: () -> Long = System::currentTimeMillis,
) : ViewModel() {
    private val activeIgnoredItems = ignoreRuleRepository.observeActive(currentTimeMillis())
    val uiState: StateFlow<SettingsUiState> = combine(
        settingsRepository.settings,
        activeIgnoredItems,
    ) { settings, ignoredItems ->
        SettingsUiState(
            largeFileThreshold = settings.largeFileThreshold,
            historyRetentionPolicy = settings.historyRetentionPolicy,
            ignoredItemCount = ignoredItems.size,
        )
    }.stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), SettingsUiState())
    init {
        viewModelScope.launch { ignoreRuleRepository.deleteExpired(currentTimeMillis()) }
    }
    fun onEvent(event: SettingsUiEvent) {
        viewModelScope.launch {
            when (event) {
```

```
                is SettingsUiEvent.SelectLargeFileThreshold ->
                    settingsRepository.setLargeFileThreshold(event.value)
                is SettingsUiEvent.SelectHistoryRetention -> {
                    settingsRepository.setHistoryRetentionPolicy(event.value)
                    scanHistoryRepository.prune(event.value)
                }
                SettingsUiEvent.ClearIgnoredItems -> ignoreRuleRepository.deleteAll()
            }
        }
    }
}
package com.cleanguard.app
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import com.cleanguard.app.navigation.CleanGuardNavHost
import com.cleanguard.app.ui.theme.CleanGuardTheme
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        val container = (application as CleanGuardApplication).container
        setContent {
            CleanGuardTheme {
                CleanGuardNavHost(container)
            }
        }
    }
}
package com.cleanguard.app.navigation
sealed class CleanGuardDestination(val route: String) {
    data object Home : CleanGuardDestination("home")
    data object Scanning : CleanGuardDestination("scanning")
    data object ScanResults : CleanGuardDestination("scan-results")
    data object CleanupDetail : CleanGuardDestination("cleanup-detail")
    data object StorageInsights : CleanGuardDestination("storage-insights")
    data object CleanupResult : CleanGuardDestination("cleanup-result")
    data object DuplicateGroups : CleanGuardDestination("duplicate-groups")
    data object ApkInsights : CleanGuardDestination("apk-insights")
    data object DownloadInsights : CleanGuardDestination("download-insights")
    data object ScanHistory : CleanGuardDestination("scan-history")
    data object CleanupHistory : CleanGuardDestination("cleanup-history")
    data object RecoveryCenter : CleanGuardDestination("recovery-center")
    data object Settings : CleanGuardDestination("settings")
    data object PermissionCenter : CleanGuardDestination("permission-center")
    data object DuplicateDetail : CleanGuardDestination("duplicate-detail/{groupId}") {
        fun createRoute(groupId: String): String = "duplicate-detail/${android.net.Uri.encode(groupId)}"
    }
}
```

```
package com.cleanguard.app.navigation
import androidx.compose.runtime.Composable
import androidx.compose.runtime.remember
import androidx.compose.ui.platform.LocalContext
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import androidx.navigation.NavType
import androidx.navigation.navArgument
import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import com.cleanguard.app.di.AppContainer
import com.cleanguard.app.feature.cleanupdetail.CleanupDetailRoute
import com.cleanguard.app.feature.cleanupdetail.CleanupDetailViewModel
import com.cleanguard.app.feature.cleanupresult.CleanupResultRoute
import com.cleanguard.app.feature.cleanupresult.CleanupResultViewModel
import com.cleanguard.app.feature.cleanuphistory.CleanupHistoryRoute
import com.cleanguard.app.feature.cleanuphistory.CleanupHistoryViewModel
import com.cleanguard.app.feature.recovery.RecoveryCenterRoute
import com.cleanguard.app.feature.recovery.RecoveryCenterViewModel
import com.cleanguard.app.feature.home.HomeRoute
import com.cleanguard.app.feature.home.HomeViewModel
import com.cleanguard.app.feature.apkinsights.ApkInsightsRoute
import com.cleanguard.app.feature.apkinsights.ApkInsightsViewModel
import com.cleanguard.app.feature.downloadinsights.DownloadInsightsRoute
import com.cleanguard.app.feature.downloadinsights.DownloadInsightsViewModel
import com.cleanguard.app.feature.duplicates.DuplicateDetailRoute
import com.cleanguard.app.feature.duplicates.DuplicateDetailViewModel
import com.cleanguard.app.feature.duplicates.DuplicateGroupsRoute
import com.cleanguard.app.feature.duplicates.DuplicateGroupsViewModel
import com.cleanguard.app.feature.insights.StorageInsightsRoute
import com.cleanguard.app.feature.insights.StorageInsightsViewModel
import com.cleanguard.app.feature.results.ScanResultsRoute
import com.cleanguard.app.feature.results.ScanResultsViewModel
import com.cleanguard.app.feature.scanning.ScanningRoute
import com.cleanguard.app.feature.scanning.ScanningViewModel
import com.cleanguard.app.feature.scanhistory.ScanHistoryRoute
import com.cleanguard.app.feature.scanhistory.ScanHistoryViewModel
import com.cleanguard.app.feature.settings.SettingsRoute
import com.cleanguard.app.feature.settings.SettingsViewModel
import com.cleanguard.app.feature.permissions.PermissionCenterRoute
import com.cleanguard.app.feature.permissions.PermissionCenterViewModel
@Composable
fun CleanGuardNavHost(container: AppContainer) {
    val navController = rememberNavController()
    val factory = remember(container) { CleanGuardViewModelFactory(container) }
    val context = LocalContext.current
    NavHost(
        navController = navController,
```

```
        startDestination = CleanGuardDestination.Home.route,
    ) {
        composable(CleanGuardDestination.Home.route) {
            HomeRoute(
                viewModel = viewModel<HomeViewModel>(factory = factory),
                onStartScan = { navController.navigate(CleanGuardDestination.Scanning.route) },
                onOpenStorageInsights = { navController.navigate(CleanGuardDestination.StorageInsights.route) },
                onOpenScanHistory = { navController.navigate(CleanGuardDestination.ScanHistory.route) },
                onOpenRecommendations = { navController.navigate(CleanGuardDestination.ScanResults.route) },
                onOpenCleanupHistory = { navController.navigate(CleanGuardDestination.CleanupHistory.route) },
                onOpenRecoveryCenter = { navController.navigate(CleanGuardDestination.RecoveryCenter.route) },
                onOpenSettings = { navController.navigate(CleanGuardDestination.Settings.route) },
                createDownloadTreeIntent = container.downloadTreeIntentFactory::create,
            )
        }
        composable(CleanGuardDestination.Settings.route) {
            SettingsRoute(
                viewModel = viewModel<SettingsViewModel>(factory = factory),
                onBack = { navController.navigateUp() },
                onOpenPermissionCenter = {
                    navController.navigate(CleanGuardDestination.PermissionCenter.route)
                },
            )
        }
        composable(CleanGuardDestination.PermissionCenter.route) {
            PermissionCenterRoute(
                viewModel = viewModel<PermissionCenterViewModel>(factory = factory),
                onBack = { navController.navigateUp() },
                createDownloadTreeIntent = container.downloadTreeIntentFactory::create,
                onOpenUsageAccessSettings = {
                    context.startActivity(container.appSettingsIntentFactory.createUsageAccessSettings())
                },
            )
        }
        composable(CleanGuardDestination.CleanupHistory.route) {
            CleanupHistoryRoute(
                viewModel = viewModel<CleanupHistoryViewModel>(factory = factory),
                onBack = { navController.navigateUp() },
                onOpenRecoveryCenter = {
                    navController.navigate(CleanGuardDestination.RecoveryCenter.route)
                },
            )
        }
        composable(CleanGuardDestination.RecoveryCenter.route) {
            RecoveryCenterRoute(
                viewModel = viewModel<RecoveryCenterViewModel>(factory = factory),
                onBack = { navController.navigateUp() },
            )
        }
        composable(CleanGuardDestination.ScanHistory.route) {
```

```
            ScanHistoryRoute(
                viewModel = viewModel<ScanHistoryViewModel>(factory = factory),
                onBack = { navController.navigateUp() },
            )
        }
        composable(CleanGuardDestination.Scanning.route) {
            ScanningRoute(
                viewModel = viewModel<ScanningViewModel>(factory = factory),
                onBack = { navController.navigateUp() },
                onViewResults = { navController.navigate(CleanGuardDestination.ScanResults.route) },
            )
        }
        composable(CleanGuardDestination.ScanResults.route) {
            ScanResultsRoute(
                viewModel = viewModel<ScanResultsViewModel>(factory = factory),
                onBack = { navController.navigateUp() },
                onViewSuggestions = { navController.navigate(CleanGuardDestination.CleanupDetail.route) },
                onOpenStorageInsights = { navController.navigate(CleanGuardDestination.StorageInsights.route) },
                onOpenDuplicateGroups = {
                    navController.navigate(CleanGuardDestination.DuplicateGroups.route)
                },
                onOpenApkInsights = {
                    navController.navigate(CleanGuardDestination.ApkInsights.route)
                },
                onOpenDownloadInsights = {
                    navController.navigate(CleanGuardDestination.DownloadInsights.route)
                },
            )
        }
        composable(CleanGuardDestination.ApkInsights.route) {
            ApkInsightsRoute(
                viewModel = viewModel<ApkInsightsViewModel>(factory = factory),
                onBack = { navController.navigateUp() },
            )
        }
        composable(CleanGuardDestination.DownloadInsights.route) {
            DownloadInsightsRoute(
                viewModel = viewModel<DownloadInsightsViewModel>(factory = factory),
                onBack = { navController.navigateUp() },
                onPreview = { file ->
                    file.contentUri?.let { uri ->
                        runCatching {
                            context.startActivity(
                                container.fileActionIntentFactory.createPreview(uri, file.mimeType),
                            )
                        }
                    }
                },
                onShare = { file ->
                    file.contentUri?.let { uri ->
```

```
                    runCatching {
                        context.startActivity(
                            android.content.Intent.createChooser(
                                container.fileActionIntentFactory.createShare(uri, file.mimeType),
                                null,
                            ),
                        )
                    }
                }
            },
            onLocate = { treeUri ->
                runCatching {
                    context.startActivity(container.fileActionIntentFactory.createLocate(treeUri))
                }
            },
        )
    }
    composable(CleanGuardDestination.DuplicateGroups.route) {
        DuplicateGroupsRoute(
            viewModel = viewModel<DuplicateGroupsViewModel>(factory = factory),
            onBack = { navController.navigateUp() },
            onOpenGroup = { groupId ->
                navController.navigate(CleanGuardDestination.DuplicateDetail.createRoute(groupId))
            },
        )
    }
    composable(
        route = CleanGuardDestination.DuplicateDetail.route,
        arguments = listOf(navArgument("groupId") { type = NavType.StringType }),
    ) { backStackEntry ->
        val groupId = backStackEntry.arguments?.getString("groupId").orEmpty()
        val detailFactory = remember(container, groupId) {
            object : ViewModelProvider.Factory {
                @Suppress("UNCHECKED_CAST")
                override fun <T : ViewModel> create(modelClass: Class<T>): T =
                    DuplicateDetailViewModel(
                        groupId = groupId,
                        duplicateSessionRepository = container.duplicateSessionRepository,
                    ) as T
            }
        }
        DuplicateDetailRoute(
            viewModel = viewModel<DuplicateDetailViewModel>(
                key = groupId,
                factory = detailFactory,
            ),
            onBack = { navController.navigateUp() },
            onReviewSelection = {
                navController.navigate(CleanGuardDestination.CleanupDetail.route)
            },
```

```
            )
        }
        composable(CleanGuardDestination.CleanupDetail.route) {
            CleanupDetailRoute(
                viewModel = viewModel<CleanupDetailViewModel>(factory = factory),
                onBack = { navController.navigateUp() },
                onCleanupFinished = {
                    navController.navigate(CleanGuardDestination.CleanupResult.route)
                },
            )
        }
        composable(CleanGuardDestination.StorageInsights.route) {
            StorageInsightsRoute(
                viewModel = viewModel<StorageInsightsViewModel>(factory = factory),
                onBack = { navController.navigateUp() },
                onOpenUsageAccessSettings = {
                    context.startActivity(container.appSettingsIntentFactory.createUsageAccessSettings())
                },
                onOpenApplicationSettings = { packageName ->
                    context.startActivity(
                        container.appSettingsIntentFactory.createApplicationDetailsSettings(packageName),
                    )
                },
            )
        }
        composable(CleanGuardDestination.CleanupResult.route) {
            CleanupResultRoute(
                viewModel = viewModel<CleanupResultViewModel>(factory = factory),
                onBack = { navController.navigateUp() },
                onReturnHome = {
                    navController.navigate(CleanGuardDestination.Home.route) {
                        popUpTo(CleanGuardDestination.Home.route) { inclusive = false }
                        launchSingleTop = true
                    }
                },
            )
        }
    }
}
package com.cleanguard.app.navigation
import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import com.cleanguard.app.di.AppContainer
import com.cleanguard.app.domain.usecase.LoadStorageOverviewUseCase
import com.cleanguard.app.domain.usecase.LoadAppStorageUseCase
import com.cleanguard.app.domain.usecase.CreateCleanupPlanUseCase
import com.cleanguard.app.domain.usecase.ApplySafetyRulesUseCase
import com.cleanguard.app.domain.usecase.ScanMediaUseCase
import com.cleanguard.app.domain.usecase.UpdateLargeFileThresholdUseCase
import com.cleanguard.app.domain.usecase.ObserveScanHistoryUseCase
```

```
import com.cleanguard.app.feature.cleanupdetail.CleanupDetailViewModel
import com.cleanguard.app.feature.cleanupresult.CleanupResultViewModel
import com.cleanguard.app.feature.cleanuphistory.CleanupHistoryViewModel
import com.cleanguard.app.feature.recovery.RecoveryCenterViewModel
import com.cleanguard.app.feature.home.HomeViewModel
import com.cleanguard.app.feature.apkinsights.ApkInsightsViewModel
import com.cleanguard.app.feature.downloadinsights.DownloadInsightsViewModel
import com.cleanguard.app.feature.duplicates.DuplicateGroupsViewModel
import com.cleanguard.app.feature.insights.StorageInsightsViewModel
import com.cleanguard.app.feature.results.ScanResultsViewModel
import com.cleanguard.app.feature.scanning.ScanningViewModel
import com.cleanguard.app.feature.scanhistory.ScanHistoryViewModel
import com.cleanguard.app.feature.settings.SettingsViewModel
import com.cleanguard.app.feature.permissions.PermissionCenterViewModel
class CleanGuardViewModelFactory(
    private val container: AppContainer,
) : ViewModelProvider.Factory {
    @Suppress("UNCHECKED_CAST")
    override fun <T : ViewModel> create(modelClass: Class<T>): T = when {
        modelClass.isAssignableFrom(HomeViewModel::class.java) ->
            HomeViewModel(
                loadStorageOverview = LoadStorageOverviewUseCase(container.storageRepository),
                permissionRepository = container.permissionRepository,
                downloadAccessRepository = container.downloadAccessRepository,
                observeScanHistory = ObserveScanHistoryUseCase(
                    container.scanHistoryRepository,
                    container.appStorageHistoryRepository,
                ),
                scanSessionRepository = container.scanRepository,
                recommendationEngine = container.recommendationEngine,
                ignoreRuleRepository = container.ignoreRuleRepository,
            )
        modelClass.isAssignableFrom(ScanHistoryViewModel::class.java) ->
            ScanHistoryViewModel(
                ObserveScanHistoryUseCase(
                    container.scanHistoryRepository,
                    container.appStorageHistoryRepository,
                ),
            )
        modelClass.isAssignableFrom(CleanupHistoryViewModel::class.java) ->
            CleanupHistoryViewModel(container.cleanupHistoryRepository)
        modelClass.isAssignableFrom(RecoveryCenterViewModel::class.java) ->
            RecoveryCenterViewModel(
                historyRepository = container.cleanupHistoryRepository,
                recoveryRepository = container.mediaRecoveryRepository,
            )
        modelClass.isAssignableFrom(ScanningViewModel::class.java) ->
            ScanningViewModel(
                scanMedia = ScanMediaUseCase(container.mediaScanner),
                applySafetyRules = ApplySafetyRulesUseCase(container.safetyRuleRepository),
```

```
                permissionRepository = container.permissionRepository,
                scanSessionRepository = container.scanRepository,
                downloadAccessRepository = container.downloadAccessRepository,
                safScanner = container.safScanner,
                settingsRepository = container.settingsRepository,
            )
        modelClass.isAssignableFrom(ScanResultsViewModel::class.java) ->
            ScanResultsViewModel(
                scanRepository = container.scanRepository,
                recommendationEngine = container.recommendationEngine,
                duplicateSessionRepository = container.duplicateSessionRepository,
                updateLargeFileThreshold = UpdateLargeFileThresholdUseCase(
                    repository = container.scanRepository,
                    applySafetyRules = ApplySafetyRulesUseCase(container.safetyRuleRepository),
                    settingsRepository = container.settingsRepository,
                ),
                ignoreRuleRepository = container.ignoreRuleRepository,
            )
        modelClass.isAssignableFrom(CleanupDetailViewModel::class.java) ->
            CleanupDetailViewModel(
                scanRepository = container.scanRepository,
                recommendationEngine = container.recommendationEngine,
                cleanupRepository = container.cleanupRepository,
                createCleanupPlan = CreateCleanupPlanUseCase(),
                initialSelectedIds = container.duplicateSessionRepository.consumeCleanupSelection()
                    .takeIf { it.isNotEmpty() },
                confirmedDuplicateIds = {
                    container.duplicateSessionRepository.groups.value
                        .flatMapTo(hashSetOf()) { group -> group.files.map { it.itemId } }
                },
                ignoreRuleRepository = container.ignoreRuleRepository,
            )
        modelClass.isAssignableFrom(DuplicateGroupsViewModel::class.java) ->
            DuplicateGroupsViewModel(
                scanRepository = container.scanRepository,
                verificationEngine = container.duplicateVerificationEngine,
                duplicateSessionRepository = container.duplicateSessionRepository,
            )
        modelClass.isAssignableFrom(ApkInsightsViewModel::class.java) ->
            ApkInsightsViewModel(
                scanRepository = container.scanRepository,
                apkAnalyzer = container.apkAnalyzer,
            )
        modelClass.isAssignableFrom(DownloadInsightsViewModel::class.java) ->
            DownloadInsightsViewModel(
                scanRepository = container.scanRepository,
                downloadAccessRepository = container.downloadAccessRepository,
                insightsEngine = container.downloadInsightsEngine,
            )
        modelClass.isAssignableFrom(StorageInsightsViewModel::class.java) ->
```

```
            StorageInsightsViewModel(
                scanRepository = container.scanRepository,
                updateLargeFileThreshold = UpdateLargeFileThresholdUseCase(
                    repository = container.scanRepository,
                    applySafetyRules = ApplySafetyRulesUseCase(container.safetyRuleRepository),
                    settingsRepository = container.settingsRepository,
                ),
                loadAppStorage = LoadAppStorageUseCase(
                    repository = container.appStorageRepository,
                    historyRepository = container.appStorageHistoryRepository,
                ),
            )
        modelClass.isAssignableFrom(CleanupResultViewModel::class.java) ->
            CleanupResultViewModel(container.cleanupRepository)
        modelClass.isAssignableFrom(SettingsViewModel::class.java) ->
            SettingsViewModel(
                settingsRepository = container.settingsRepository,
                scanHistoryRepository = container.scanHistoryRepository,
                ignoreRuleRepository = container.ignoreRuleRepository,
            )
        modelClass.isAssignableFrom(PermissionCenterViewModel::class.java) ->
            PermissionCenterViewModel(
                permissionRepository = container.permissionRepository,
                downloadAccessRepository = container.downloadAccessRepository,
                appStorageRepository = container.appStorageRepository,
            )
        else -> error("Unknown ViewModel class: ${modelClass.name}")
    } as T
}
package com.cleanguard.app.platform.intents
import android.content.Intent
import android.net.Uri
import android.provider.Settings
class AppSettingsIntentFactory {
    fun createUsageAccessSettings(): Intent = Intent(Settings.ACTION_USAGE_ACCESS_SETTINGS)
    fun createApplicationDetailsSettings(packageName: String): Intent = Intent(
        Settings.ACTION_APPLICATION_DETAILS_SETTINGS,
        Uri.fromParts("package", packageName, null),
    )
}
package com.cleanguard.app.platform.intents
import android.content.Intent
import android.provider.DocumentsContract
import androidx.core.net.toUri
class DownloadTreeIntentFactory {
    fun create(): Intent = Intent(Intent.ACTION_OPEN_DOCUMENT_TREE).apply {
        addFlags(
            Intent.FLAG_GRANT_READ_URI_PERMISSION or
                Intent.FLAG_GRANT_PERSISTABLE_URI_PERMISSION or
                Intent.FLAG_GRANT_PREFIX_URI_PERMISSION,
```

```
        )
        putExtra(
            DocumentsContract.EXTRA_INITIAL_URI,
            "content://com.android.externalstorage.documents/document/primary%3ADownload".toUri(),
        )
    }
}
package com.cleanguard.app.platform.intents
import android.content.Intent
import android.provider.DocumentsContract
import androidx.core.net.toUri
class FileActionIntentFactory {
    fun createPreview(contentUri: String, mimeType: String?): Intent =
        Intent(Intent.ACTION_VIEW).apply {
            setDataAndType(contentUri.toUri(), mimeType ?: "*/*")
            addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
        }
    fun createShare(contentUri: String, mimeType: String?): Intent =
        Intent(Intent.ACTION_SEND).apply {
            type = mimeType ?: "*/*"
            putExtra(Intent.EXTRA_STREAM, contentUri.toUri())
            addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
        }
    fun createLocate(treeUri: String?): Intent = Intent(Intent.ACTION_OPEN_DOCUMENT).apply {
        addCategory(Intent.CATEGORY_OPENABLE)
        type = "*/*"
        addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
        treeUri?.takeIf { it.isNotBlank() }?.let {
            putExtra(DocumentsContract.EXTRA_INITIAL_URI, it.toUri())
        }
    }
}
package com.cleanguard.app.platform.permissions
import android.content.Context
import android.content.pm.PackageManager
import android.os.Build
import androidx.core.content.ContextCompat
import com.cleanguard.app.domain.model.MediaPermissionStatus
import com.cleanguard.app.domain.model.MediaPermissionOverview
import com.cleanguard.app.domain.model.PermissionGrantState
import com.cleanguard.app.domain.repository.PermissionRepository
class PermissionManager(
    context: Context,
) : PermissionRepository {
    private val appContext = context.applicationContext
    override fun getMediaPermissionStatus(): MediaPermissionStatus =
        MediaPermissionPolicy.resolve(Build.VERSION.SDK_INT) { permission ->
            ContextCompat.checkSelfPermission(appContext, permission) == PackageManager.PERMISSION_GRANTED
        }
    override fun permissionsForRequest(): List<String> =
```

```
        MediaPermissionPolicy.permissionsForRequest(Build.VERSION.SDK_INT)
    override fun getMediaPermissionOverview(): MediaPermissionOverview =
        MediaPermissionPolicy.overview(Build.VERSION.SDK_INT) { permission ->
            ContextCompat.checkSelfPermission(appContext, permission) == PackageManager.PERMISSION_GRANTED
        }
}
internal object MediaPermissionPolicy {
    private const val READ_EXTERNAL_STORAGE = "android.permission.READ_EXTERNAL_STORAGE"
    private const val READ_MEDIA_IMAGES = "android.permission.READ_MEDIA_IMAGES"
    private const val READ_MEDIA_VIDEO = "android.permission.READ_MEDIA_VIDEO"
    private const val READ_MEDIA_AUDIO = "android.permission.READ_MEDIA_AUDIO"
    private const val READ_MEDIA_VISUAL_USER_SELECTED =
        "android.permission.READ_MEDIA_VISUAL_USER_SELECTED"
    fun permissionsForRequest(sdkInt: Int): List<String> = when {
        sdkInt >= Build.VERSION_CODES.UPSIDE_DOWN_CAKE -> listOf(
            READ_MEDIA_IMAGES,
            READ_MEDIA_VIDEO,
            READ_MEDIA_AUDIO,
            READ_MEDIA_VISUAL_USER_SELECTED,
        )
        sdkInt >= Build.VERSION_CODES.TIRAMISU -> listOf(
            READ_MEDIA_IMAGES,
            READ_MEDIA_VIDEO,
            READ_MEDIA_AUDIO,
        )
        else -> listOf(READ_EXTERNAL_STORAGE)
    }
    fun resolve(
        sdkInt: Int,
        isGranted: (String) -> Boolean,
    ): MediaPermissionStatus {
        val fullAccessPermissions = when {
            sdkInt >= Build.VERSION_CODES.TIRAMISU -> listOf(
                READ_MEDIA_IMAGES,
                READ_MEDIA_VIDEO,
                READ_MEDIA_AUDIO,
            )
            else -> listOf(READ_EXTERNAL_STORAGE)
        }
        if (fullAccessPermissions.all(isGranted)) {
            return MediaPermissionStatus.FULL
        }
        val hasSelectedVisualAccess =
            sdkInt >= Build.VERSION_CODES.UPSIDE_DOWN_CAKE &&
                isGranted(READ_MEDIA_VISUAL_USER_SELECTED)
        val hasAnyMediaAccess = fullAccessPermissions.any(isGranted)
        return if (hasSelectedVisualAccess || hasAnyMediaAccess) {
            MediaPermissionStatus.PARTIAL
        } else {
            MediaPermissionStatus.NONE
```

```
        }
    }
    fun overview(
        sdkInt: Int,
        isGranted: (String) -> Boolean,
    ): MediaPermissionOverview {
        if (sdkInt < Build.VERSION_CODES.TIRAMISU) {
            val state = if (isGranted(READ_EXTERNAL_STORAGE)) {
                PermissionGrantState.AUTHORIZED
            } else {
                PermissionGrantState.NOT_AUTHORIZED
            }
            return MediaPermissionOverview(state, state, state)
        }
        val selected = sdkInt >= Build.VERSION_CODES.UPSIDE_DOWN_CAKE &&
            isGranted(READ_MEDIA_VISUAL_USER_SELECTED)
        fun visualState(permission: String): PermissionGrantState = when {
            isGranted(permission) -> PermissionGrantState.AUTHORIZED
            selected -> PermissionGrantState.PARTIAL
            else -> PermissionGrantState.NOT_AUTHORIZED
        }
        return MediaPermissionOverview(
            photos = visualState(READ_MEDIA_IMAGES),
            videos = visualState(READ_MEDIA_VIDEO),
            audio = if (isGranted(READ_MEDIA_AUDIO)) {
                PermissionGrantState.AUTHORIZED
            } else {
                PermissionGrantState.NOT_AUTHORIZED
            },
        )
    }
}
package com.cleanguard.app.ui.components
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.material3.TopAppBar
import androidx.compose.runtime.Composable
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun CleanGuardTopBar(
    title: String,
    onBack: (() -> Unit)? = null,
) {
    TopAppBar(
        title = { Text(title) },
        navigationIcon = {
            if (onBack != null) {
                TextButton(onClick = onBack) {
                    Text("??")
```

```
                }
            }
        },
    )
}
package com.cleanguard.app.ui.components
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.ColumnScope
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
@Composable
fun InfoCard(
    title: String,
    modifier: Modifier = Modifier,
    content: @Composable ColumnScope.() -> Unit,
) {
    Card(
        modifier = modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface),
    ) {
        Column(
            modifier = Modifier.padding(18.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp),
        ) {
            Text(title, style = MaterialTheme.typography.titleMedium)
            content()
        }
    }
}
package com.cleanguard.app.ui.components
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
@Composable
fun MetricTile(
```

```
    label: String,
    value: String,
    modifier: Modifier = Modifier,
) {
    Surface(
        modifier = modifier,
        color = MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.55f),
        shape = MaterialTheme.shapes.medium,
    ) {
        Column(
            modifier = Modifier.fillMaxWidth().padding(14.dp),
            verticalArrangement = Arrangement.spacedBy(4.dp),
        ) {
            Text(label, style = MaterialTheme.typography.bodyMedium, color = MaterialTheme.colorScheme.onSurfaceVariant)
            Text(value, style = MaterialTheme.typography.titleMedium)
        }
    }
}
package com.cleanguard.app.ui.theme
import androidx.compose.ui.graphics.Color
val GuardGreen = Color(0xFF176B5B)
val GuardGreenDark = Color(0xFF87D5C2)
val GuardGreenContainer = Color(0xFFD2F0E8)
val GuardBlue = Color(0xFF37618A)
val GuardAmber = Color(0xFF8A5A00)
val GuardRed = Color(0xFF9B4145)
val GuardInk = Color(0xFF18201E)
val GuardSurface = Color(0xFFF7FAF9)
val GuardSurfaceDark = Color(0xFF101514)
package com.cleanguard.app.ui.theme
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.darkColorScheme
import androidx.compose.material3.lightColorScheme
import androidx.compose.runtime.Composable
import androidx.compose.ui.graphics.Color
private val LightColors = lightColorScheme(
    primary = GuardGreen,
    onPrimary = Color.White,
    primaryContainer = GuardGreenContainer,
    onPrimaryContainer = Color(0xFF002019),
    secondary = GuardBlue,
    onSecondary = Color.White,
    tertiary = GuardAmber,
    error = GuardRed,
    background = GuardSurface,
    onBackground = GuardInk,
    surface = Color.White,
    onSurface = GuardInk,
    surfaceVariant = Color(0xFFE4ECE9),
```

```
    onSurfaceVariant = Color(0xFF414946),
    outline = Color(0xFF707976),
)
private val DarkColors = darkColorScheme(
    primary = GuardGreenDark,
    onPrimary = Color(0xFF00382E),
    primaryContainer = Color(0xFF005143),
    onPrimaryContainer = GuardGreenContainer,
    secondary = Color(0xFFA2C9F2),
    onSecondary = Color(0xFF043354),
    tertiary = Color(0xFFFFBA47),
    error = Color(0xFFFFB3B5),
    background = GuardSurfaceDark,
    onBackground = Color(0xFFDFE4E1),
    surface = Color(0xFF171C1B),
    onSurface = Color(0xFFDFE4E1),
    surfaceVariant = Color(0xFF414946),
    onSurfaceVariant = Color(0xFFC0C9C5),
    outline = Color(0xFF8A938F),
)
@Composable
fun CleanGuardTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    content: @Composable () -> Unit,
) {
    MaterialTheme(
        colorScheme = if (darkTheme) DarkColors else LightColors,
        typography = CleanGuardTypography,
        content = content,
    )
}
package com.cleanguard.app.ui.theme
import androidx.compose.material3.Typography
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.sp
val CleanGuardTypography = Typography(
    displaySmall = TextStyle(
        fontFamily = FontFamily.SansSerif,
        fontWeight = FontWeight.SemiBold,
        fontSize = 34.sp,
        lineHeight = 40.sp,
    ),
    headlineMedium = TextStyle(
        fontFamily = FontFamily.SansSerif,
        fontWeight = FontWeight.SemiBold,
        fontSize = 26.sp,
        lineHeight = 32.sp,
    ),
```

```
    titleLarge = TextStyle(
        fontFamily = FontFamily.SansSerif,
        fontWeight = FontWeight.SemiBold,
        fontSize = 20.sp,
        lineHeight = 26.sp,
    ),
    titleMedium = TextStyle(
        fontFamily = FontFamily.SansSerif,
        fontWeight = FontWeight.Medium,
        fontSize = 16.sp,
        lineHeight = 22.sp,
    ),
    bodyLarge = TextStyle(
        fontFamily = FontFamily.SansSerif,
        fontWeight = FontWeight.Normal,
        fontSize = 16.sp,
        lineHeight = 24.sp,
    ),
    bodyMedium = TextStyle(
        fontFamily = FontFamily.SansSerif,
        fontWeight = FontWeight.Normal,
        fontSize = 14.sp,
        lineHeight = 21.sp,
    ),
    labelLarge = TextStyle(
        fontFamily = FontFamily.SansSerif,
        fontWeight = FontWeight.SemiBold,
        fontSize = 14.sp,
        lineHeight = 20.sp,
    ),
)
package com.cleanguard.app.ui.util
import java.time.Instant
import java.time.ZoneId
import java.time.format.DateTimeFormatter
import java.util.Locale
private const val KIB = 1024.0
private const val MIB = KIB * 1024.0
private const val GIB = MIB * 1024.0
fun Long.formatBytes(): String = when {
    this >= GIB -> String.format(Locale.SIMPLIFIED_CHINESE, "%.1f GB", this / GIB)
    this >= MIB -> String.format(Locale.SIMPLIFIED_CHINESE, "%.0f MB", this / MIB)
    this >= KIB -> String.format(Locale.SIMPLIFIED_CHINESE, "%.0f KB", this / KIB)
    else -> "$this B"
}
fun Long.formatDate(): String = DateTimeFormatter.ofPattern("yyyy-MM-dd")
    .withLocale(Locale.SIMPLIFIED_CHINESE)
    .withZone(ZoneId.systemDefault())
    .format(Instant.ofEpochMilli(this))
package com.cleanguard.app.ui.util
```

```
import androidx.annotation.StringRes
import com.cleanguard.app.R
import com.cleanguard.app.domain.recommendation.RecommendationPriority
import com.cleanguard.app.domain.recommendation.RecommendationReason
@StringRes
fun RecommendationPriority.labelResource(): Int = when (this) {
    RecommendationPriority.HIGH -> R.string.recommendation_priority_high
    RecommendationPriority.MEDIUM -> R.string.recommendation_priority_medium
    RecommendationPriority.LOW -> R.string.recommendation_priority_low
    RecommendationPriority.NONE -> R.string.recommendation_priority_none
}
@StringRes
fun RecommendationReason.labelResource(): Int = when (this) {
    RecommendationReason.CONFIRMED_DUPLICATE -> R.string.recommendation_reason_confirmed_duplicate
    RecommendationReason.OLD_APK -> R.string.recommendation_reason_old_apk
    RecommendationReason.OLD_DOWNLOAD -> R.string.recommendation_reason_old_download
    RecommendationReason.LONG_UNUSED_DOWNLOAD -> R.string.recommendation_reason_long_unused_download
    RecommendationReason.LARGE_SCREEN_RECORDING -> R.string.recommendation_reason_large_screen_recording
    RecommendationReason.OLD_SCREENSHOT -> R.string.recommendation_reason_old_screenshot
    RecommendationReason.LARGE_FILE -> R.string.recommendation_reason_large_file
    RecommendationReason.VERY_LARGE_FILE -> R.string.recommendation_reason_very_large_file
    RecommendationReason.APP_STORAGE_GROWTH -> R.string.recommendation_reason_app_storage_growth
    RecommendationReason.RECENT_FILE -> R.string.recommendation_reason_recent_file
    RecommendationReason.USER_IGNORED -> R.string.recommendation_reason_user_ignored
}
package com.cleanguard.app.ui.util
import androidx.annotation.StringRes
import com.cleanguard.app.R
import com.cleanguard.app.domain.model.MediaScanExecutionType
@StringRes
fun MediaScanExecutionType.labelResource(): Int = when (this) {
    MediaScanExecutionType.FULL -> R.string.scan_type_full
    MediaScanExecutionType.INCREMENTAL -> R.string.scan_type_incremental
    MediaScanExecutionType.FULL_RESYNC -> R.string.scan_type_full_resync
    MediaScanExecutionType.API_29_FULL -> R.string.scan_type_api29_full
}
```